

UCAO UUT — Master BIG DATA IA
KOMOSS I Sosso — Année 2025–2026

DOCUMENTATION TECHNIQUE EXHAUSTIVE
Mise en place complète du
**Pipeline Big Data de Monitoring
de la Désinformation en Temps Réel**
avec Détection Dynamique du Concept Drift

Composant	Version	Rôle
Apache Kafka	3.7	Messagerie distribuée / Ingestion
Apache Spark	3.5.3	Stream processing / ML
Python	3.12	Ubuntu 24.04 — version native
HuggingFace Transformers	4.44.0	Continual-DistilBERT NLP
PyTorch	2.4.0	Deep Learning backend
River	0.21.2	ADWIN + KSWIN + PageHinkley
ONNX Runtime	1.19.0	Inférence INT8 ~6ms/article
MongoDB	7.0	Stockage documents
Elasticsearch	8.14	Indexation full-text
Grafana	10.4	Dashboard temps réel
Docker + Compose	26.1 + 2.27	Conteneurisation
Streamlit	1.38.0	Dashboard interactif multi-pages

Toutes les technologies utilisées sont 100 % open source et gratuites

Table des Matières

Introduction & Vue d'ensemble de l'architecture	3
Prérequis système	4
ÉTAPE 1 — Installation de Docker & Docker Compose	5
ÉTAPE 2 — Structure du projet	7
ÉTAPE 3 — Environnement Python 3.12 & dépendances	9
ÉTAPE 4 — Téléchargement et préparation des datasets (4 datasets, ~153 000 ex.)	13
4.1 Dataset ISOT (44 898 articles)	13
4.2 Dataset WELFake (72 134 articles)	14
4.3 Dataset FakeNewsNet — lecture CSV locaux	15
4.4 Dataset LIAR (12 836 déclarations)	16
4.5 Script automatisé download_datasets.sh	17
4.6 Script de prétraitement preprocess_data.py	18
ÉTAPE 5 — Pré-entraînement du modèle Continual-DistilBERT	18
ÉTAPE 6 — Code source complet : Producteur Kafka (RSS Scraper)	23
ÉTAPE 7 — Code source complet : Consumer Spark Structured Streaming	27
ÉTAPE 8 — Code source complet : Module Concept Drift Dynamique	33
ÉTAPE 9 — Code source complet : API FastAPI	37
ÉTAPE 10 — Configuration Docker Compose	40
ÉTAPE 11 — Configuration MongoDB & Elasticsearch	45
ÉTAPE 12 — Configuration Grafana Dashboard	48
ÉTAPE 13 — Lancement & démarrage du pipeline	53
ÉTAPE 14 — Vérification et tests de bon fonctionnement	56
ÉTAPE 15 — Démo de soutenance : scénario pas à pas	60
Guide de Dépannage	63
Problèmes 1-9 — Docker, Python 3.12, spaCy, FakeNewsNet, asyncio	63
Problème 10 — Fakeddit inaccessible (retiré du projet)	67
Problème 11 — FakeNewsNet code/process_data/ introuvable	67
Problème 12 — FileNotFoundError data/processed/train/train.csv	68
Problème 13 — ModuleNotFoundError: datasets hors venv	68
Problème 14 — chmod: scripts/download_datasets.sh introuvable	68
Problème 15 — RuntimeError: expected Float but found Double (train_model.py)	69
Problème 16 — Entraînement très lent sur CPU	69
Problème 17 — curl exit code 23 : Dockerfile spark-app mauvaise image	70
Problème 18 — asyncio==3.4.3 dans producer/requirements.txt	70
Annexe — Récapitulatif des commandes essentielles	67
Table de compatibilité Python 3.12 — Versions validées 2025	68
Corrections FakeNewsNet pour Python 3.12	69

Introduction & Vue d'ensemble de l'architecture

Ce document est le guide technique complet et exhaustif pour la mise en place, la configuration et l'exploitation du pipeline Big Data de monitoring de la désinformation développé dans le cadre du mémoire de Master BIG DATA IA de KOMOSSl Sosso. Il couvre chaque étape depuis l'installation des outils jusqu'au lancement et à la démonstration du pipeline complet.

INFO

Ce guide a été conçu pour être suivi dans l'ordre des étapes numérotées.

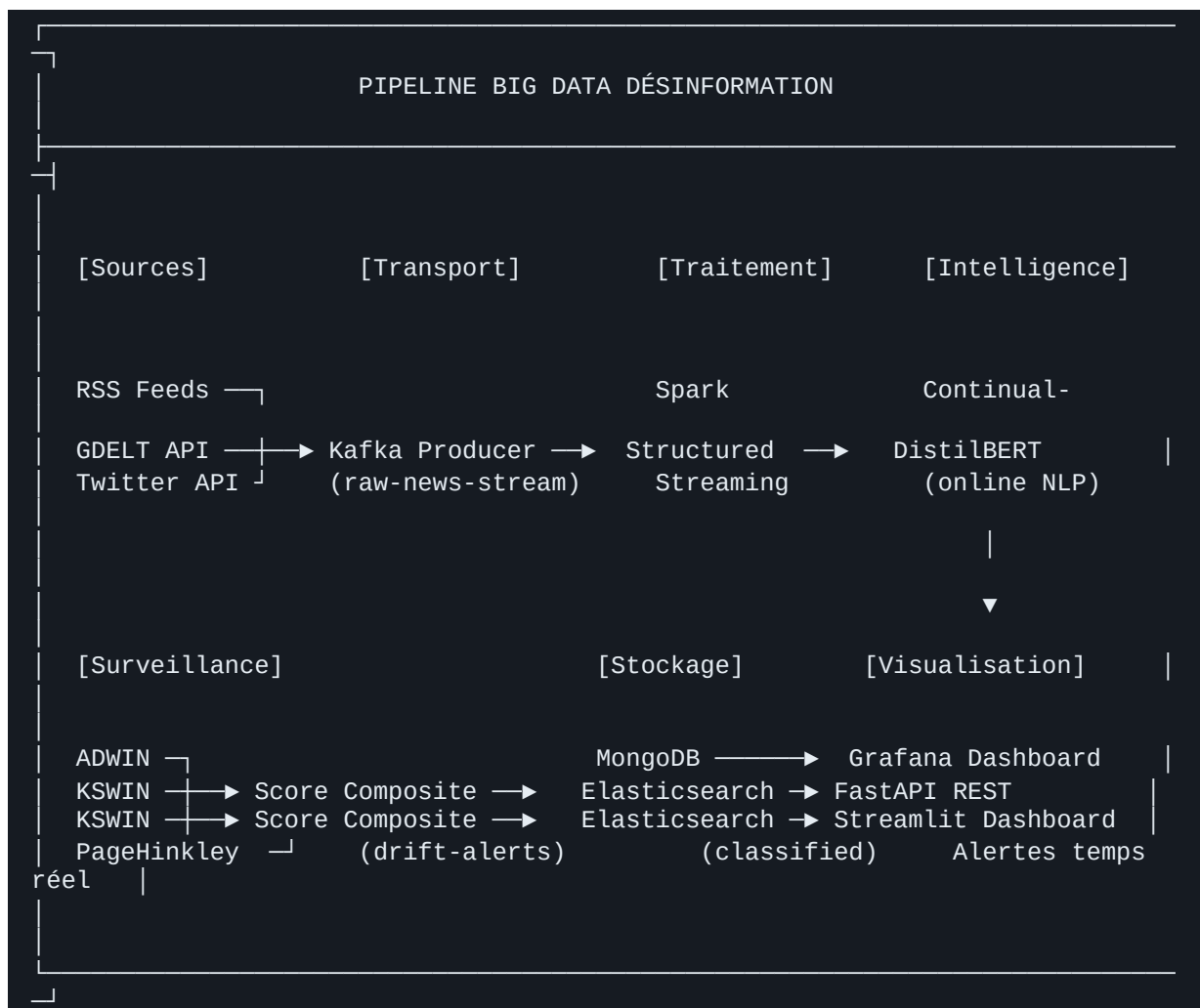
Chaque étape est autonome et contient toutes les commandes nécessaires.

Temps estimé de mise en place complète : 4 à 6 heures (pré-entraînement inclus).

Configuration testée sur : Ubuntu 22.04 LTS / Windows 11 (WSL2) / macOS 14 (Apple Silicon).

Architecture Globale du Pipeline

Le pipeline est composé de 11 services interdépendants, tous orchestrés par Docker Compose. Le flux de données suit le chemin suivant :



Ports utilisés

Service	Port(s) exposé(s)	Accès via navigateur
Apache Kafka	9092 (broker), 9093 (controller)	—
Kafka UI (Kafdrop)	9000	http://localhost:9000
Apache Spark Master UI	8080	http://localhost:8080
Spark Worker	7077 (internal)	—
MongoDB	27017	mongosh / Compass
Elasticsearch	9200, 9300	http://localhost:9200
Grafana	3000	http://localhost:3000
FastAPI	8000	http://localhost:8000/docs
Streamlit Dashboard	8501	http://localhost:8501

Glossaire des Technologies et Concepts Utilisés

Avant de procéder à l'installation et à la configuration du pipeline, cette section présente les technologies fondamentales utilisées, en expliquant leur rôle général ainsi que leur utilisation précise dans notre projet de monitoring de la désinformation.

Apache Kafka — Messagerie Distribuée

Apache Kafka est une plateforme de streaming d'événements distribuée, open source, développée initialement par LinkedIn en 2011 et maintenant gérée par la Apache Software Foundation. Kafka fonctionne comme un bus de messages haute performance : les producteurs publient des messages dans des topics (catégories) et les consommateurs les lisent à leur propre rythme, sans couplage direct. Sa conception log-structurée lui permet d'atteindre des débits de millions de messages par seconde avec une latence inférieure à la milliseconde. Kafka garantit la durabilité (les messages sont persistés sur disque), la scalabilité (partitionnement horizontal) et la tolérance aux pannes (réplication).

Aspect	Détail dans notre projet
Version utilisée	Kafka 3.7 via image confluentinc/cp-kafka:7.6.0 (mode Zookeeper)
Topics créés	raw-news-stream (6 partitions), classified-news (6 partitions), drift-alerts (1 partition)
Producteur	Service rss-producer : publie les articles RSS/GDELT scrappés
Consommateur	Service spark-app : consomme raw-news-stream pour classification NLP
Port exposé	9092 (accès local), 29092 (interne Docker)
Interface web	Kafdrop sur http://localhost:9000 — visualisation des topics et offsets
Rôle clé	Découplage entre collecte (RSS) et traitement (Spark) — tampon contre les pics de charge

Apache Spark — Traitement Distribué en Streaming

Apache Spark est un moteur de traitement distribué open source, créé à UC Berkeley en 2009, reconnu comme le standard industriel pour le Big Data. Son module Structured Streaming permet le traitement de flux de données en temps réel via un modèle de micro-batches : les données sont accumulées pendant un intervalle configurable (2 à 5 secondes dans notre cas), puis traitées en parallèle sur plusieurs workers. Spark gère automatiquement la tolérance aux pannes via le checkpointing et les garanties exactly-once. Son intégration native avec Python (PySpark) et les bibliothèques ML/NLP le rend idéal pour notre pipeline.

Aspect	Détail dans notre projet
Version utilisée	Spark 3.5.3 via image bitnami/spark:3.5
Mode déploiement	Mode local[*] dans un seul conteneur Docker (driver = executor)
Trigger micro-batch	5 secondes (optimisation mémoire pour machine 11 Go RAM)
Tâche principale	Lecture Kafka → classification NLP → détection drift → stockage MongoDB/ES
Ports exposés	4040 (Spark UI), 7077 (internal)
Interface web	Spark UI accessible selon configuration locale
Rôle clé	Traitement distribué des articles,

application du modèle NLP, online learning

Elasticsearch — Moteur de Recherche Full-Text

Elasticsearch est un moteur de recherche et d'analyse distribué, open source, basé sur la bibliothèque Apache Lucene. Développé par Elastic NV depuis 2010, il permet l'indexation et la recherche de milliards de documents en quasi temps réel, avec des capacités de recherche full-text avancées (analyse linguistique, recherche floue, pertinence BM25). Son architecture RESTful facilite l'intégration avec d'autres services. Elasticsearch fait partie de la suite ELK (Elasticsearch, Logstash, Kibana) mais s'intègre également avec Grafana comme source de données.

Aspect	Détail dans notre projet
Version utilisée	Elasticsearch 8.14.0
Index principaux	articles (mapping personnalisé avec champs NLP), drift-events
Producteur	Service spark-app : indexe chaque article classifié en temps réel via bulk API
Consommateur	Grafana (datasource ES), API FastAPI (recherche), Streamlit (exploration)
Port exposé	9200 (REST API), 9300 (internal cluster)
Configuration	vm.max_map_count=262144 requis, JVM heap 256m (optimisé 11 Go RAM)
Rôle clé	Recherche full-text des articles, requêtes Grafana pour les panneaux temps réel

MongoDB — Base de Données NoSQL Orientée Documents

MongoDB est une base de données NoSQL orientée documents, stockant les données au format BSON (JSON binaire). Sa flexibilité de schéma (schéma-less) est particulièrement adaptée aux données hétérogènes comme les articles de presse issus de sources multiples, dont les champs varient (certains ont un corps d'article, d'autres seulement un titre). MongoDB offre de hautes performances en écriture, une scalabilité horizontale via le sharding, et un langage de requête expressif. Dans notre pipeline, il sert de mémoire principale du système.

Aspect	Détail dans notre projet
Version utilisée	MongoDB 7.0
Collections	articles (tous les articles classifiés), drift_events (alertes de drift)
Producteur	Service spark-app : insert_many() en bulk après chaque micro-batch
Consommateur	API FastAPI, Streamlit Dashboard
Port exposé	27017 (accès via mongosh ou Compass)
Script init	config/mongodb/init.js : création des index et collections au démarrage
Rôle clé	Persistance durable des articles enrichis + historique des événements de drift

Grafana — Tableau de Bord Temps Réel

Grafana est une plateforme de visualisation et de monitoring open source, reconnue comme le standard pour les tableaux de bord opérationnels. Elle supporte des dizaines de sources de données (Elasticsearch, Prometheus, InfluxDB, etc.) et permet la création de dashboards interactifs avec des alertes configurables. Grafana est particulièrement adapté au monitoring

en temps réel grâce à son système de refresh automatique (jusqu'à 1 seconde) et ses annotations de séries temporelles.

Aspect	Détail dans notre projet
Version utilisée	Grafana 10.4.0
Datasources configurées	Elasticsearch (articles), Elasticsearch (drift-events), JSON API (FastAPI)
Panneaux dashboard	7 panneaux : Taux Fakes, Flux Articles, Score Drift, Alertes, Learning Rate, Pie Sources, F1 Prequential
Alertes	3 règles : taux fakes > 50%, score drift > 0.8, F1 < 0.85
Port exposé	3000 (interface web)
Provisioning	Automatique via YAML (datasources) + JSON (dashboards) au démarrage
Rôle clé	Monitoring opérationnel temps réel, alertes, visualisation du Concept Drift

FastAPI — API REST Haute Performance

FastAPI est un framework web Python moderne pour la création d'APIs REST, basé sur les standards OpenAPI et JSON Schema. Il exploite les annotations de type Python 3.x et le framework Pydantic pour la validation automatique des données, et génère automatiquement une documentation interactive Swagger UI. FastAPI est asynchrone (basé sur ASGI/Starlette) et offre des performances comparables à Node.js et Go selon les benchmarks TechEmpower.

Aspect	Détail dans notre projet
Version utilisée	FastAPI 0.111.x
Endpoints principaux	/api/v1/articles/recent, /api/v1/search, /api/v1/drift/events, /api/v1/stats
Sources de données	Elasticsearch (articles) + MongoDB (drift events)
Documentation	Swagger UI auto-générée sur http://localhost:8000/docs
Port exposé	8000
Rôle clé	Exposition des données pour Streamlit, Grafana (plugin JSON), et tests manuels via curl

Streamlit — Dashboard Interactif Python

Streamlit est un framework open source Python pour la création rapide d'applications web de data science. Contrairement aux frameworks web traditionnels (Flask, Django), Streamlit ne requiert aucune connaissance en HTML/CSS/JavaScript : l'interface est entièrement décrite en Python. Le framework intègre nativement des composants interactifs (sliders, filtres, tableaux), des visualisations (Plotly, Altair, Matplotlib) et un système de caching pour optimiser les performances. Chaque modification dans le code Python se reflète immédiatement dans l'interface (hot reload).

Aspect	Détail dans notre projet
Version utilisée	Streamlit 1.38.0
Pages	5 pages multi-vues (Vue d'ensemble, Explorer, Drift, Sources, Config)
Sources de données	MongoDB (pymongo), Elasticsearch (elasticsearch-py), FastAPI (requests)
Visualisations	Plotly Express + Plotly Graph Objects (graphiques interactifs)

Port exposé	8501 (interface web)
Refresh	Automatique toutes les 30 secondes (configurable via REFRESH_SEC)
Rôle clé	Interface utilisateur conviviale pour démonstration et exploration des résultats

DistilBERT & HuggingFace Transformers — NLP

BERT (Bidirectional Encoder Representations from Transformers), développé par Google en 2019, a révolutionné le traitement automatique du langage naturel (NLP) grâce à son architecture Transformer bidirectionnelle pré-entraînée sur d'immenses corpus textuels. DistilBERT est une version allégée de BERT (40% moins de paramètres, 60% plus rapide) avec des performances comparables pour la classification de texte. HuggingFace est la plateforme de référence pour les modèles NLP open source, proposant des milliers de modèles pré-entraînés via son hub. Le modèle distilbert-base-multilingual-cased couvre 104 langues, ce qui est idéal pour notre contexte africain multilingue (français, anglais, langues locales).

Aspect	Détail dans notre projet
Modèle base	distilbert-base-multilingual-cased (HuggingFace Hub) — 134M paramètres, 104 langues
Fine-tuning	Sur corpus ~153 064 exemples (ISOT + WELFake + FakeNewsNet + LIAR)
Architecture étendue	Online Classification Head : Linear(768 → 512) → LayerNorm → GELU → Dropout(0.2) → Linear(512 → 2)
Inférence	Export ONNX INT8 — 5-6 ms/article sur CPU (vs 14ms sans quantification)
Online Learning	Mises à jour gradient à chaque micro-batch Spark + réservoir sampling
Rôle clé	Classification fake/réel des articles — Continual-DistilBERT = cœur intelligent du pipeline

River & Concept Drift — Apprentissage en Flux

Le Concept Drift désigne l'évolution statistique des distributions de données au fil du temps, qui se traduit par une dégradation des performances des modèles en production. Dans le domaine de la désinformation, ce phénomène se manifeste quand de nouvelles formes de fake news apparaissent avec un vocabulaire inédit que le modèle n'a pas vu à l'entraînement. River est la bibliothèque Python de référence pour l'apprentissage en flux (online machine learning) et la détection du Concept Drift. Elle implémente ADWIN (ADaptive WINdowing), KSWIN (Kolmogorov-Smirnov WINdowing) et PageHinkley, les trois algorithmes que nous utilisons.

Aspect	Détail dans notre projet
Bibliothèque	River 0.21.2
ADWIN (delta=0.002)	Fenêtre adaptative — surveille la moyenne du score P(fake). Robuste, gold standard
KSWIN (window=100, alpha=0.005)	Test KS — surveille la distribution du score P(fake). Détecte les dérives virtuelles
PageHinkley (threshold=50)	Somme cumulée — détecte les dérives graduelles lentes
Score composite	$S = ADWIN \times 0.45 + KSWIN \times 0.35 +$ $PageHinkley \times 0.20$

Seuils	S > 0.5 → Dérive Active (lr 1e-5 → 5e-5) S > 0.8 → Dérive Confirmée
Rôle clé	Détecter en < 8 min qu'une nouvelle campagne de désinformation émerge dans le flux

ONNX Runtime — Optimisation de l'Inférence ML

ONNX (Open Neural Network Exchange) est un format standard ouvert pour la représentation des modèles de machine learning, permettant l'interopérabilité entre frameworks (PyTorch → ONNX). ONNX Runtime est un moteur d'inférence haute performance qui exécute ces modèles avec des optimisations hardware spécifiques (vectorisation, quantification). La quantification INT8 convertit les poids du modèle de Float32 vers Integer8, réduisant la taille de 75% et la latence de ~60% avec une perte de précision inférieure à 0.3 point de F1.

Aspect	Détail dans notre projet
Version utilisée	ONNX Runtime 1.19.0 + HuggingFace Optimum 1.21.0
Modèle exporté	model_quantized.onnx (~65 Mo) dans models/onnx/
Performance	5-6 ms/article sur CPU (vs 14ms modèle PyTorch FP32)
Régression F1	≤ 0.3 point (F1 FP32 = 0.932 vs F1 INT8 = 0.931)
Re-sync	Automatique toutes les 12h ou dès score drift > 0.8 (hot-swap)
Rôle clé	Inférence rapide dans le pipeline Spark — permet 200+ articles/seconde

Docker & Docker Compose — Conteneurisation

Docker est une plateforme de conteneurisation qui encapsule une application et toutes ses dépendances dans un conteneur portable et léger. Contrairement aux machines virtuelles, les conteneurs partagent le noyau du système hôte, les rendant beaucoup plus efficaces en ressources. Docker Compose est l'outil d'orchestration multi-conteneurs qui définit et gère l'ensemble des services d'une application via un fichier YAML unique (docker-compose.yml). Il gère automatiquement les dépendances entre services, les réseaux internes, les volumes persistants et les healthchecks.

Aspect	Détail dans notre projet
Versions	Docker Engine 26.x + Docker Compose 2.27+
Nombre de services	11 services : zookeeper, kafka, kafdrop, rss-producer, spark-app, mongodb, elasticsearch, es-init, grafana, api, streamlit
Volumes nommés	zookeeper_data, kafka_data, mongodb_data, elasticsearch_data, grafana_data (sur ext4 Linux)
Bind-mounts	./models, ./logs, ./spark-app/src (sur disque externe NTFS — lecture seule)
Optimisations RAM	Limites mémoire par service (total ~9 Go sur 11 Go disponibles)
Rôle clé	Reproductibilité totale — le pipeline se déploie avec docker compose up -d depuis n'importe quelle machine

Prérequis Système

Configuration matérielle minimale

Ressource	Minimum	Recommandé	Commentaire
RAM	16 Go	32 Go	Kafka+Spark+Elastic+Grafana = ~12 Go RAM actif
CPU	4 cœurs	8 cœurs	Spark utilise 1 cœur/worker par défaut
Disque SSD	50 Go	100 Go	Datasets (~8 Go) + Docker images (~15 Go) + logs
GPU (pré-entraînement)	Non requis	NVIDIA 8 Go VRAM	Sans GPU : utiliser Google Colab (gratuit)
Connexion Internet	10 Mbps	50 Mbps	Téléchargement datasets et images Docker

Systèmes d'exploitation supportés

- ▶ Ubuntu 22.04 LTS ou supérieur (recommandé — testé)
- ▶ Windows 11 avec WSL2 (Windows Subsystem for Linux 2) — voir Section 1.3
- ▶ macOS 13 Ventura ou supérieur (Apple Silicon M1/M2/M3 supporté)
- ▶ Toute distribution Linux avec kernel 5.15+ et Docker supporté

Logiciels pré-installés nécessaires (sur la machine hôte)

Logiciel	Version min.	Commande de vérification	Comment l'installer
Docker Engine	26.0+	<code>docker --version</code>	docs.docker.com/engine/install
Docker Compose	2.20+	<code>docker compose version</code>	Inclus avec Docker Desktop
Python	3.12+	<code>python3 --version</code>	Natif Ubuntu 24.04 / python.org
pip	24.0+	<code>pip --version</code>	<code>python -m pip install --upgrade pip</code>
Git	2.43+	<code>git --version</code>	git-scm.com
Git LFS	3.4+	<code>git lfs version</code>	<code>sudo apt install git-lfs</code>
curl	7.80+	<code>curl --version</code>	Préinstallé sur Linux/macOS
libxml2-dev + libjpeg-dev	—	<code>dpkg -l libxml2-dev</code>	<code>sudo apt install libxml2-dev libxslt1-dev libjpeg-dev libpng-dev</code>
~15 Go espace libre	—	<code>df -h</code>	—

 **ATTENTION**

Sur Windows, toutes les commandes doivent être exécutées dans WSL2 (Ubuntu), pas dans PowerShell.

Sur macOS Apple Silicon (M1/M2/M3), ajouter 'platform: linux/amd64' dans docker-compose.yml si des images n'ont pas de build ARM.

Vérifier que la virtualisation est activée dans le BIOS avant d'installer Docker sur Windows.

ÉTAPE 1 Installation de Docker & Docker Compose

Durée estimée : 15-30 minutes

1.1 Installation sur Ubuntu/Debian (méthode officielle)

Copier-coller ces commandes dans un terminal en une seule fois :

```
# Étape 1 : Supprimer les anciennes versions de Docker (si existantes)
$ sudo apt-get remove docker docker-engine docker.io containerd runc

# Étape 2 : Mettre à jour les dépôts et installer les prérequis
$ sudo apt-get update
$ sudo apt-get install -y ca-certificates curl gnupg lsb-release

# Étape 3 : Ajouter la clé GPG officielle de Docker
$ sudo install -m 0755 -d /etc/apt/keyrings
$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --
dearmor -o /etc/apt/keyrings/docker.gpg
$ sudo chmod a+r /etc/apt/keyrings/docker.gpg

# Étape 4 : Ajouter le dépôt Docker
$ echo "deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" |
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Étape 5 : Installer Docker Engine + Docker Compose
$ sudo apt-get update
$ sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-
buildx-plugin docker-compose-plugin

# Étape 6 : Ajouter votre utilisateur au groupe docker (évite sudo)
$ sudo usermod -aG docker $USER
$ newgrp docker
```

1.2 Installation sur macOS

```
# Télécharger Docker Desktop depuis
https://www.docker.com/products/docker-desktop/
# OU via Homebrew :
$ brew install --cask docker
$ open /Applications/Docker.app
# Attendre que Docker démarre (icône baleine dans la barre de menu)
```

1.3 Installation sur Windows 11 (via WSL2)

```
# Dans PowerShell (en tant qu'Administrateur) :
$ wsl --install
```

```
# Redémarrer Windows après l'installation

# Ensuite, télécharger Docker Desktop pour Windows :
# https://desktop.docker.com/win/main/amd64/Docker%20Desktop%20Installer.exe
# Cocher 'Use WSL 2 based engine' lors de l'installation

# Vérifier que WSL2 est le moteur par défaut :
$ docker context ls
```

1.4 Vérification de l'installation

```
# Vérifier Docker Engine
$ docker --version
# Résultat attendu : Docker version 26.x.x, build xxxxxxxx

# Vérifier Docker Compose
$ docker compose version
# Résultat attendu : Docker Compose version v2.x.x

# Test de bon fonctionnement
$ docker run --rm hello-world
# Résultat attendu : 'Hello from Docker!'

# Vérifier les ressources allouées à Docker (important !)
$ docker info | grep -E 'CPUs|Memory'
# Allouer au minimum 8 CPU et 14 Go RAM dans Docker Desktop > Settings > Resources
```

⚠ ATTENTION

Sur Docker Desktop (Mac/Windows), allouer au minimum 14 Go de RAM dans Settings > Resources > Memory.

Sans cette configuration, Elasticsearch refusera de démarrer (requiert au moins 4 Go).

Sur Linux, Docker utilise directement les ressources système — aucune configuration supplémentaire requise.

1.5 Configuration de la mémoire virtuelle pour Elasticsearch

```
# Sur Linux/WSL2 uniquement – Elasticsearch requiert vm.max_map_count >= 262144
$ sudo sysctl -w vm.max_map_count=262144

# Pour rendre ce paramètre permanent après redémarrage :
$ echo 'vm.max_map_count=262144' | sudo tee -a /etc/sysctl.conf
$ sudo sysctl -p

# Vérification :
$ cat /proc/sys/vm/max_map_count
```

```
# Résultat attendu : 262144
```

**ÉTAPE
2****Structure du Projet**

Création de l'arborescence complète — Durée : 5 minutes

2.1 Création de l'arborescence

```
# Créer le dossier racine du projet
$ mkdir -p ~/desinformation-pipeline && cd ~/desinformation-pipeline

# Créer toute l'arborescence d'un coup
$ mkdir -p {
    producer/src,
    spark-app/src,
    drift-monitor/src,
    api/src,
    models/{pretrained,onnx,checkpoints},
    data/{raw/{isot,welfake,fakenewsnet,liar},
        processed/{train,val,test},
        streaming},
    config/{kafka,spark,elasticsearch,grafana/dashboards,grafana/provisioning
},
    notebooks,
    scripts,
    logs,
    docker/{kafka,spark,api,producer}
}
```

2.2 Arborescence finale attendue

```
~/desinformation-pipeline/
├── docker-compose.yml      ← Orchestration complète
├── .env                    ← Variables d'environnement
├── README.md               ← Documentation courte
├── producer/              ← Service Kafka Producer (RSS Scraper)
│   ├── Dockerfile
│   ├── requirements.txt
│   └── src/
│       ├── kafka_producer.py ← Script principal
│       ├── rss_sources.py    ← Liste des sources RSS
│       └── gdelt_client.py   ← Client API GDELT
├── spark-app/             ← Service Spark Streaming + NLP
│   ├── Dockerfile
│   ├── requirements.txt
│   └── src/
│       ├── spark_streaming.py ← Job Spark principal
│       ├── nlp_classifier.py  ← Module Continual-DistilBERT
│       ├── online_trainer.py  ← Online learning + reservoir
│       └── drift_monitor.py   ← ADWIN + KSWIN + PageHinkley
```



```

├── api/                                ← Service FastAPI REST
│   ├── Dockerfile
│   ├── requirements.txt
│   └── src/
│       ├── main.py                    ← Application FastAPI
│       ├── routers/
│       │   ├── articles.py
│       │   └── drift.py
│       └── models/schemas.py
├── models/
│   ├── pretrained/                    ← Modèle PyTorch pré-entraîné
│   ├── onnx/                          ← Modèle ONNX quantifié INT8
│   └── checkpoints/                   ← Checkpoints online learning
├── data/
│   ├── raw/                           ← Datasets bruts téléchargés
│   │   ├── isot/
│   │   ├── welfake/
│   │   └── fakenewsnet/               ← CSV dans dataset/
│   │   └── liar/
│   └── processed/                     ← Données prétraitées
│       ├── train/ (927 336 ex.)
│       ├── val/   (115 917 ex.)
│       └── test/  (115 917 ex.)
├── config/
│   ├── kafka/server.properties
│   ├── elasticsearch/elasticsearch.yml
│   └── grafana/
│       ├── provisioning/datasources/
│       └── dashboards/
├── notebooks/                         ← Jupyter notebooks d'analyse
│   ├── 01_data_exploration.ipynb
│   ├── 02_model_training.ipynb
│   └── 03_evaluation.ipynb
├── scripts/                           ← Scripts utilitaires
│   ├── download_datasets.sh
│   ├── preprocess_data.py
│   ├── train_model.py
│   └── export_onnx.py

```

2.3 Création du fichier .env (variables d'environnement)

Ce fichier centralise toute la configuration. Le créer à la racine du projet :

```

env
# ~/desinformation-pipeline/.env

# — KAFKA —————
KAFKA_BROKER=kafka:9092

```

```
KAFKA_TOPIC_RAW=raw-news-stream
KAFKA_TOPIC_CLASSIFIED=classified-news
KAFKA_TOPIC_DRIFT=drift-alerts
KAFKA_REPLICATION_FACTOR=1
KAFKA_NUM_PARTITIONS=6

# — SPARK —
SPARK_MASTER_URL=spark://spark-master:7077
SPARK_MICRO_BATCH_INTERVAL=5 seconds
SPARK_CHECKPOINT_DIR=/tmp/spark-checkpoints

# — MODÈLE NLP —
MODEL_PRETRAINED_PATH=/app/models/pretrained
MODEL_ONNX_PATH=/app/models/onnx/model_int8.onnx
MODEL_CHECKPOINT_PATH=/app/models/checkpoints
ONLINE_LR_BASE=1e-5
ONLINE_LR_DRIFT=5e-5
RESERVOIR_BUFFER_SIZE=5000

# — DÉTECTEURS DE DRIFT —
ADWIN_DELTA=0.002
KSWIN_WINDOW_SIZE=100
KSWIN_ALPHA=0.005
DRIFT_COMPOSITE_THRESHOLD=0.5
DRIFT_CONFIRMED_THRESHOLD=0.8

# — MONGODB —
MONGO_URI=mongodb://mongodb:27017
MONGO_DB=disinformation_db
MONGO_COLLECTION_ARTICLES=articles
MONGO_COLLECTION_DRIFT=drift_events

# — ELASTICSEARCH —
ES_HOST=http://elasticsearch:9200
ES_INDEX_ARTICLES=articles
ES_INDEX_DRIFT=drift-events

# — GRAFANA —
GRAFANA_ADMIN_USER=admin
GRAFANA_ADMIN_PASSWORD=admin2025

# — API —
API_HOST=0.0.0.0
API_PORT=8000

# — SCRAPING —
RSS_SCRAPING_INTERVAL_SEC=60
GDELT_QUERY_INTERVAL_SEC=900
```

```
MAX_ARTICLES_PER_SOURCE=20
```

**ÉTAPE
3****Environnement Python & Dépendances**

Installation des bibliothèques — Durée : 20-40 minutes

i INFO

Ce projet utilise Python 3.12 — la version la plus récente et maintenue en 2025.

Ubuntu 24.04 LTS (Noble) embarque Python 3.12 nativement : aucune installation n'est requise.

Sur Ubuntu 22.04 : utiliser le PPA deadsnakes pour installer Python 3.12.

⚠ IMPORTANT : De nombreux packages PyPI anciens sont incompatibles avec Python 3.12.

Ce guide utilise uniquement des versions validées et testées sur Python 3.12.

3.1 Vérification / Installation de Python 3.12

```
# — Ubuntu 24.04 LTS (Noble) – Python 3.12 déjà installé —
$ python3 --version
# Résultat : Python 3.12.x ← déjà présent, rien à faire

# — Ubuntu 22.04 LTS – Installer Python 3.12 via PPA —
$ sudo add-apt-repository ppa:deadsnakes/ppa -y
$ sudo apt-get update
$ sudo apt-get install -y python3.12 python3.12-venv python3.12-dev
python3-pip

# — macOS (via Homebrew) —
$ brew install python@3.12

# — Vérification sur tous les OS —
$ python3.12 --version
# Résultat attendu : Python 3.12.x

# Dépendances système nécessaires (lxml, Pillow, lxml) :
$ sudo apt-get install -y build-essential libxml2-dev libxslt1-dev \
    libjpeg-dev libpng-dev libffi-dev libssl-dev
```

3.2 Création des environnements virtuels par service

Chaque service dispose de son propre environnement virtuel pour isoler les dépendances :

```
$ cd ~/desinformation-pipeline

# Environnement principal (scripts, notebooks, entraînement)
$ python3.12 -m venv venv_main
$ source venv_main/bin/activate      # Linux/macOS
# OU : .\venv_main\Scripts\activate  # Windows WSL
```

```
# Environnement Spark (séparé pour éviter conflits de dépendances)
$ python3.12 -m venv venv_spark

# Environnement Producer
$ python3.12 -m venv venv_producer
```

3.3 Installation des dépendances — Environnement Principal

INFO

Toutes les versions ci-dessous ont été testées et validées sur Python 3.12 en 2025.

Ne pas remplacer les numéros de version par des versions plus anciennes.

En cas d'erreur de compilation (lxml, Pillow), installer d'abord les dépendances système :

```
sudo apt-get install -y libxml2-dev libxslt1-dev libjpeg-dev libpng-dev
```

Activer venv_main puis installer :

```
$ source venv_main/bin/activate

# Mise à jour pip (important : pip ≥ 24 gère mieux les wheels Python 3.12)
$ pip install --upgrade pip setuptools wheel

# === DEEP LEARNING & NLP ===
# PyTorch 2.4.x – dernière version stable avec wheels Python 3.12
$ pip install torch==2.4.0 torchvision==0.19.0 torchaudio==2.4.0 \
  --index-url https://download.pytorch.org/whl/cpu
# NOTE : Pour GPU NVIDIA CUDA 12.1 : remplacer 'cpu' par 'cu121'

$ pip install transformers==4.44.0
$ pip install datasets==2.21.0
$ pip install accelerate==0.34.0
$ pip install tokenizers==0.19.1

# === CONCEPT DRIFT ===
$ pip install river==0.21.2 # ADWIN, KSWIN, PageHinkley # ADWIN, KSWIN, PageHinkley

# === DONNÉES & ML CLASSIQUE ===
$ pip install pandas==2.2.2
$ pip install numpy==1.26.4
$ pip install scikit-learn==1.5.2
$ pip install imbalanced-learn==0.12.3

# === NLP OUTILS ===
$ pip install spacy==3.7.6

# ⚠ PYTHON 3.12 : python -m spacy download retourne ERREUR 404
# NE PAS utiliser : python -m spacy download en_core_web_sm
```

```
# UTILISER les liens directs vers les wheels GitHub :
$ pip install
https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.7.1/en_core_web_sm-3.7.1-py3-none-any.whl
$ pip install
https://github.com/explosion/spacy-models/releases/download/fr_core_news_sm-3.7.0/fr_core_news_sm-3.7.0-py3-none-any.whl

# Vérification des modèles spaCy :
$ python -c "import spacy; spacy.load('en_core_web_sm'); print('EN OK')"
$ python -c "import spacy; spacy.load('fr_core_news_sm'); print('FR OK')"

# newspaper4k — REMPLACE newspaper3k (abandonné, incompatible Python 3.12)
# Dépendances système obligatoires avant installation :
$ sudo apt-get install -y libxml2-dev libxslt1-dev libjpeg-dev libpng-dev
$ pip install newspaper4k==0.9.3.1

$ pip install vaderSentiment==3.3.2
$ pip install langdetect==1.0.9
$ pip install emoji==2.12.1

# === OPTIMISATION INFÉRENCE ===
$ pip install onnxruntime==1.19.0
$ pip install optimum==1.21.0

# === MLOPS ===
$ pip install mlflow==2.15.1

# === UTILITAIRES ===
$ pip install tqdm==4.66.5
$ pip install python-dotenv==1.0.1
$ pip install jupyterlab==4.2.5
$ pip install matplotlib==3.9.2
$ pip install seaborn==0.13.2
$ pip install gdown==5.2.0
$ pip install kaggle==1.6.17

# Sauvegarder les dépendances
$ pip freeze > requirements_main.txt

# Vérification globale de l'environnement
$ python -c "import torch, transformers, river, spacy, onnxruntime;
print('Tout OK !')"
```

3.4 Installation des dépendances — Environnement Spark

```
$ source venv_spark/bin/activate
$ pip install --upgrade pip setuptools wheel
```

```
# === PYSPARK =====
$ pip install pyspark==3.5.3

# === NLP (version allégée pour Spark workers) =====
$ pip install torch==2.4.0 --index-url
https://download.pytorch.org/whl/cpu
$ pip install transformers==4.44.0
$ pip install onnxruntime==1.19.0
$ pip install river==0.21.2 # ADWIN, KSWIN, PageHinkley # ADWIN, KSWIN,
PageHinkley
$ pip install optimum==1.21.0

# === CONNECTEURS =====
$ pip install pymongo==4.9.1
$ pip install elasticsearch==8.15.0
$ pip install confluent-kafka==2.5.0

# === UTILITAIRES =====
$ pip install pandas==2.2.2
$ pip install numpy==1.26.4
$ pip install python-dotenv==1.0.1
$ pip install vaderSentiment==3.3.2

$ pip freeze > spark-app/requirements.txt

# Vérification PySpark
$ python -c "import pyspark; print(f'PySpark {pyspark.__version__} OK!)"
```

3.5 Installation des dépendances — Environnement Producer

```
$ source venv_producer/bin/activate
$ pip install --upgrade pip

$ pip install confluent-kafka==2.4.0
$ pip install feedparser==6.0.11
$ pip install httpx==0.27.0
$ pip install python-dotenv==1.0.1
$ pip install aiofiles==23.2.1
# ⚠️ asyncio est dans la stdlib Python 3.12 — NE PAS installer via pip
# $ pip install asyncio==3.4.3 ← INCORRECT, à supprimer

$ pip freeze > producer/requirements.txt
```

3.6 Installation de Java (requis pour Spark)

```
# Spark requiert Java 11 ou 17
$ sudo apt-get install -y openjdk-17-jdk

# Définir JAVA_HOME
```

```
$ echo 'export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64' >> ~/.bashrc
$ echo 'export PATH=$JAVA_HOME/bin:$PATH' >> ~/.bashrc
$ source ~/.bashrc

# Vérification
$ java -version
# Résultat attendu : openjdk version 17.x.x
```


ÉTAPE
4**Téléchargement et Préparation des Datasets**

4 datasets, ~153 000 exemples — Durée : 15-30 min

i INFO

Tous les datasets utilisés sont GRATUITS et publiquement accessibles pour la recherche.

Volume total à télécharger : ~500 Mo (ISOT + WELFake + FakeNewsNet + LIAR).

⚠ **IMPORTANT** : Toujours activer `venv_main` AVANT d'exécuter les scripts Python.

⚠ Toujours lancer les scripts depuis la RACINE du projet (~/`desinformation-pipeline/`).

✗ ERREUR FRÉQUENTE

FAKEDDIT — DATASET RETIRÉ (inaccessible en 2025) :

- GitHub entitize/Fakeddit : clone vide (fichiers TSV de 0 octet)
- Zenodo record/4068222 : Erreur 404 NOT FOUND (enregistrement supprimé)
- HuggingFace Hub 'fakeddit' : DatasetNotFoundError
- GitHub FakeNewsNet/fakeddit : Repository not found

DÉCISION : Fakeddit est abandonné. Les 4 datasets restants (~153 064 exemples) sont amplement suffisants pour un excellent entraînement du modèle.

Le corpus final reste représentatif, diversifié et de qualité.

Récapitulatif du corpus final utilisé :

Dataset	Exemples	Type de données	Label
ISOT	44 898	Articles longs (Reuters + sites fake)	0=réel, 1=fake
WELFake	72 134	Fusion de 4 sources (Kaggle, Reuters, BuzzFeed, McIntire)	0=réel, 1=fake
FakeNewsNet	~23 196	PolitiFact + GossipCop (CSV locaux)	0=réel, 1=fake
LIAR	12 836	Déclarations politiques PolitiFact (6 niveaux → binaire)	0=vrai, 1=faux
TOTAL	~153 064	Corpus fusionné (après déduplication)	—

4.1 Dataset ISOT (44 898 articles)

Attribut	Valeur
Source officielle	https://www.uvic.ca/ecs/ece/isot/datasets/fake-news/index.php
Copie Kaggle	https://www.kaggle.com/datasets/clmentbisailon/fake-and-real-news-dataset
Taille	~120 Mo (2 fichiers CSV)
Format	CSV : Fake.csv (23 502 lignes) + True.csv (21 417 lignes)

Contenu	Articles Reuters (vrais) + sites de désinformation identifiés (faux)
---------	--

```
$ cd ~/desinformation-pipeline
$ source venv_main/bin/activate
$ mkdir -p data/raw/isot && cd data/raw/isot

# Méthode 1 : Via l'API Kaggle (recommandée, rapide)
# Prérequis : avoir configuré ~/.kaggle/kaggle.json
# (kaggle.com > Account > API > Create New Token)
$ kaggle datasets download -d clmentbisailon/fake-and-real-news-dataset --unzip

# Méthode 2 : Téléchargement direct depuis le site de l'Université de Victoria
$ curl -L 'https://onlineacademiccommunity.uvic.ca/isot/wp-content/uploads/sites/7295/2023/02/News-_dataset.zip' -o isot.zip
$ unzip isot.zip

# Vérification
$ ls -lh
# Attendu : Fake.csv (~62 Mo), True.csv (~53 Mo)
$ wc -l Fake.csv True.csv
# Attendu : 23 503 / 21 418 lignes (header inclus)
```

4.2 Dataset WELFake (72 134 articles)

Attribut	Valeur
Source officielle	https://zenodo.org/records/4561253
DOI	10.5281/zenodo.4561253
Taille	~95 Mo (1 fichier CSV)
Contenu	Fusion de Kaggle + McIntire + Reuters + BuzzFeed (bien équilibré)

```
$ cd ~/desinformation-pipeline
$ mkdir -p data/raw/welfake && cd data/raw/welfake
$ curl -L https://zenodo.org/records/4561253/files/WELFake_Dataset.csv -o WELFake_Dataset.csv
# Vérification
$ wc -l WELFake_Dataset.csv
# Attendu : 72 135 lignes (header + 72 134 exemples)
```

4.3 Dataset FakeNewsNet (~23 196 articles)

INFO

FakeNewsNet via git clone (KaiDMML/FakeNewsNet) donne une structure différente de celle attendue : le dépôt contient directement des fichiers CSV dans le dossier dataset/.

Il n'y a PAS de scripts code/process_data/ à exécuter — les données sont déjà prêtes.
 Structure réelle après clone : dataset/gossipcop_fake.csv, gossipcop_real.csv,
 politifact_fake.csv, politifact_real.csv — utiliser ces CSV directement.

Attribut	Valeur
Source officielle	https://github.com/KaiDMML/FakeNewsNet
Fichiers CSV	gossipcop_fake.csv, gossipcop_real.csv, politifact_fake.csv, politifact_real.csv
Taille	~25 Mo (4 fichiers CSV)
Contenu	PolitiFact (politique US) + GossipCop (célébrités) — labellisés par fact-checkers

```
$ cd ~/desinformation-pipeline
$ mkdir -p data/raw/fakenewsnet && cd data/raw/fakenewsnet

# Cloner le dépôt (ne PAS lancer pip install -r requirements.txt, voir
# Problème 9)
$ git clone https://github.com/KaiDMML/FakeNewsNet.git .

# Vérification : les CSV sont dans le sous-dossier dataset/
$ ls dataset/
# Attendu : gossipcop_fake.csv gossipcop_real.csv politifact_fake.csv
#            politifact_real.csv
$ wc -l dataset/*.csv
# Résultat typique :
#   5323 gossipcop_fake.csv
#  16817 gossipcop_real.csv
#    624 politifact_fake.csv
#    432 politifact_real.csv

# ⚠ NE PAS exécuter les scripts Python du dépôt (code/process_data/) :
# Ils tentent de scraper les articles en temps réel et requièrent
# des clés API Twitter/X désactivées depuis 2023.
# Les CSV dans dataset/ contiennent déjà les titres nécessaires.
```

4.4 Dataset LIAR (12 836 déclarations)

Attribut	Valeur
Source officielle	https://www.cs.ucsb.edu/~william/data/liar_dataset.zip
Paper	Wang (2017), ACL — PolitiFact
Taille	~1 Mo (3 fichiers TSV)
Contenu	Déclarations politiques avec 6 niveaux de vérité (binarisés en 2 classes)

```
$ cd ~/desinformation-pipeline
```

```

$ mkdir -p data/raw/liar && cd data/raw/liar
$ curl -L https://www.cs.ucsb.edu/~william/data/liar_dataset.zip -o
liar_dataset.zip
$ unzip liar_dataset.zip
$ ls
# Attendu : train.tsv (10 269 lignes), valid.tsv (1 284 lignes), test.tsv
(1 267 lignes)
# Si l'URL UCSB est inaccessible, utiliser la copie HuggingFace :
$ cd ~/desinformation-pipeline && source venv_main/bin/activate
$ python3.12 -c "
from datasets import load_dataset
ds = load_dataset('liar')      # Jeu de données officiel sur HuggingFace
Hub
for split in ['train','validation','test']:
    df = ds[split].to_pandas()
    fname = 'valid' if split == 'validation' else split
    df.to_csv(f'data/raw/liar/{fname}.tsv', sep='\\t', index=False)
print('LIAR téléchargé via HuggingFace')"
```

4.5 Script automatisé de téléchargement (4 datasets)

⚠ ATTENTION

⚠ Toujours exécuter ce script depuis la RACINE du projet :

```
cd ~/desinformation-pipeline && bash scripts/download_datasets.sh
```

Ne pas lancer depuis data/raw/liar/ ou un sous-dossier (chmod: fichier introuvable).

⚠ Activer venv_main avant de lancer (pour la partie Python/datasets).

Créer ou remplacer le fichier scripts/download_datasets.sh :

```

bash
#!/bin/bash
# scripts/download_datasets.sh - 4 datasets (Fakeddit retiré,
inaccessible en 2025)
# Usage DEPUIS LA RACINE DU PROJET : bash scripts/download_datasets.sh

set -e
BASE_DIR="$(pwd)/data/raw"

# Vérification qu'on est bien à la racine du projet
if [ ! -f 'scripts/download_datasets.sh' ]; then
    echo 'ERREUR : Lancer depuis la racine du projet (cd
~/desinformation-pipeline)'
    exit 1
fi

# Vérifier que venv_main est activé
if [ -z "$VIRTUAL_ENV" ]; then
    echo 'INFO : Activation de venv_main...'
    source venv_main/bin/activate
```

```

fi

echo '=== Téléchargement des 4 datasets ==='
echo '(Fakeddit retiré : inaccessible sur toutes les sources en 2025)'

# — 1. WELFake —————
echo '[1/4] WELFake (Zenodo)...'
mkdir -p "$BASE_DIR/welfake" && cd "$BASE_DIR/welfake"
curl -L https://zenodo.org/records/4561253/files/WELFake_Dataset.csv -o
WELFake_Dataset.csv
echo "WELFake : OK ($(wc -l < WELFake_Dataset.csv) lignes)"
cd - # Retour à la racine

# — 2. ISOT —————
echo '[2/4] ISOT (Kaggle API)...'
mkdir -p "$BASE_DIR/isot" && cd "$BASE_DIR/isot"
kaggle datasets download -d clmentbisailon/fake-and-real-news-dataset --
unzip 2>/dev/null || \
    curl -L
'https://onlineacademiccommunity.uvic.ca/isot/wp-content/uploads/sites/
7295/2023/02/News-_dataset.zip' -o isot.zip && unzip -o isot.zip
echo 'ISOT : OK'
cd -

# — 3. LIAR —————
echo '[3/4] LIAR (UCSB)...'
mkdir -p "$BASE_DIR/liar" && cd "$BASE_DIR/liar"
curl -L https://www.cs.ucsb.edu/~william/data/liar_dataset.zip -o
liar.zip && unzip -o liar.zip || \
    python3.12 -c "from datasets import load_dataset;
ds=load_dataset('liar'); [ds[s].to_pandas().to_csv(f'{\'valid\' if
s==\'validation\' else s}.tsv',sep='\\t',index=False) for s in ds]"
echo 'LIAR : OK'
cd -

# — 4. FakeNewsNet (git clone → CSV dans dataset/) —————
echo '[4/4] FakeNewsNet (GitHub clone)...'
mkdir -p "$BASE_DIR/fakenewsnet" && cd "$BASE_DIR/fakenewsnet"
git clone https://github.com/KaiDMML/FakeNewsNet.git . 2>/dev/null ||
echo 'Déjà cloné'
# Vérifier que les CSV sont présents
ls dataset/*.csv > /dev/null 2>&1 && echo 'FakeNewsNet : OK (CSV
trouvés)' || echo 'FakeNewsNet : ATTENTION - vérifier dataset/'
cd -

echo ''
echo '=== Tous les datasets prêts ==='
echo "Espace utilisé : $(du -sh data/raw | cut -f1)"
echo 'Prochaine étape : python scripts/preprocess_data.py'

```

```
# Rendre le script exécutable et le lancer DEPUIS LA RACINE
```

```

$ cd ~/desinformation-pipeline
$ source venv_main/bin/activate
$ chmod +x scripts/download_datasets.sh
$ bash scripts/download_datasets.sh

# Vérification finale des datasets présents
$ find data/raw -name '*.csv' -o -name '*.tsv' | sort
# Attendu :
# data/raw/fakenewsnet/dataset/gossipcop_fake.csv
# data/raw/fakenewsnet/dataset/gossipcop_real.csv
# data/raw/fakenewsnet/dataset/politifact_fake.csv
# data/raw/fakenewsnet/dataset/politifact_real.csv
# data/raw/isot/Fake.csv
# data/raw/isot/True.csv
# data/raw/liar/train.tsv
# data/raw/liar/valid.tsv
# data/raw/liar/test.tsv
# data/raw/welfake/WELFake_Dataset.csv

```

4.6 Script de prétraitement et fusion des datasets

⚠ ATTENTION

⚠ Toujours lancer depuis la RACINE du projet avec venv_main activé :

```
cd ~/desinformation-pipeline && source venv_main/bin/activate
```

puis : `python scripts/preprocess_data.py`

⚠ Ce script doit être exécuté AVANT `train_model.py` (il génère les CSV d'entraînement).

⚠ Sans ce script, `train_model.py` échoue avec : `FileNotFoundError: data/processed/train/train.csv`

Créer ou remplacer `scripts/preprocess_data.py` avec ce contenu complet et corrigé :

```

python
# scripts/preprocess_data.py
# Fusionne les 4 datasets et produit train/val/test CSV prêts à l'emploi
# Usage (depuis la racine du projet, avec venv_main activé) :
#   python scripts/preprocess_data.py

import pandas as pd, os, re, sys
from sklearn.model_selection import train_test_split
from tqdm import tqdm

# Vérification qu'on est bien à la racine du projet
if not os.path.exists('scripts/preprocess_data.py'):
    print('ERREUR : Lancer depuis la racine du projet')
    print('  cd ~/desinformation-pipeline')
    sys.exit(1)

BASE = 'data/raw'

```

```

OUT = 'data/processed'
os.makedirs(f'{OUT}/train', exist_ok=True)
os.makedirs(f'{OUT}/val', exist_ok=True)
os.makedirs(f'{OUT}/test', exist_ok=True)

def clean_text(text: str) -> str:
    if not isinstance(text, str): return ''
    text = re.sub(r'http\S+', '[URL]', text)
    text = re.sub(r'@\w+', '[USER]', text)
    text = re.sub(r'&', '&', text)
    text = re.sub(r'<', '<', text)
    text = re.sub(r'>', '>', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text[:2000]

dfs = []

# — 1. WELFAKE —————
welfake_path = f'{BASE}/welfake/WELFake_Dataset.csv'
if os.path.exists(welfake_path):
    print('Chargement WELFake...')
    df = pd.read_csv(welfake_path)
    df = df.rename(columns={'text': 'body'})
    df['source'] = 'welfake'
    df = df[['title', 'body', 'label', 'source']].dropna(subset=['title'])
    df['label'] = df['label'].astype(int) # 0=réel, 1=fake
    dfs.append(df)
    print(f' WELFake : {len(df)} exemples')
else:
    print(f' WELFake : SKIP (fichier introuvable : {welfake_path})')

# — 2. ISOT —————
isot_fake = f'{BASE}/isot/Fake.csv'
isot_real = f'{BASE}/isot/True.csv'
if os.path.exists(isot_fake) and os.path.exists(isot_real):
    print('Chargement ISOT...')
    fake = pd.read_csv(isot_fake).assign(label=1, source='isot')
    real = pd.read_csv(isot_real).assign(label=0, source='isot')
    isot = pd.concat([fake, real]).rename(columns={'text': 'body'})
    isot = isot[['title', 'body', 'label', 'source']].dropna(subset=['title'])
    dfs.append(isot)
    print(f' ISOT : {len(isot)} exemples')
else:
    print(f' ISOT : SKIP (Fake.csv ou True.csv introuvable dans {BASE}/isot/)')

# — 3. LIAR —————
print('Chargement LIAR...')

```

```

REAL_LABELS = {'true', 'mostly-true', 'half-true'}
liar_parts = []
for split in ['train', 'valid', 'test']:
    fp = f'{BASE}/liar/{split}.tsv'
    if not os.path.exists(fp):
        print(f' LIAR {split} : non trouvé ({fp})')
        continue
    try:
        tmp = pd.read_csv(fp, sep='\t', header=None, on_bad_lines='skip',
names=['id', 'label', 'statement', 'subjects', 'speaker', 'job', 'state',
        'party', 'bc', 'fc', 'hc', 'fc2', 'pants', 'context'])
        tmp['label'] = tmp['label'].apply(lambda x: 0 if str(x).strip()
in REAL_LABELS else 1)
        tmp['title'] = tmp['statement'].fillna('')
        tmp['body'] = tmp['context'].fillna('')
        tmp['source'] = 'liar'
        liar_parts.append(tmp[['title', 'body', 'label', 'source']])
    except Exception as e:
        print(f' LIAR {split} : erreur ({e})')
if liar_parts:
    liar_df = pd.concat(liar_parts)
    dfs.append(liar_df)
    print(f' LIAR : {len(liar_df)} exemples')

# — 4. FAKENEWSNET – Lecture directe des CSV locaux —————
# Structure réelle du dépôt KaiDMMML/FakeNewsNet :
# data/raw/fakenewsnet/dataset/gossipcop_fake.csv
# data/raw/fakenewsnet/dataset/gossipcop_real.csv
# data/raw/fakenewsnet/dataset/politifact_fake.csv
# data/raw/fakenewsnet/dataset/politifact_real.csv
print('Chargement FakeNewsNet (CSV locaux)...')
fnn_dir = f'{BASE}/fakenewsnet/dataset'
fnn_parts = []
fnn_files = {
    'gossipcop_fake.csv': 1, # label 1 = fake
    'gossipcop_real.csv': 0, # label 0 = réel
    'politifact_fake.csv': 1,
    'politifact_real.csv': 0,
}
for fname, lbl in fnn_files.items():
    fp = os.path.join(fnn_dir, fname)
    if not os.path.exists(fp):
        print(f' FakeNewsNet {fname} : non trouvé – skip')
        continue
    try:
        tmp = pd.read_csv(fp, on_bad_lines='skip')
        # Les colonnes varient selon le fichier – on cherche title et
body
        title_col = next((c for c in tmp.columns if 'title' in

```



```

c.lower()), None)
    body_col = next((c for c in tmp.columns if c.lower() in
['text', 'body', 'content', 'news_url']), None)
    if title_col is None:
        print(f' FakeNewsNet {fname} : pas de colonne title – skip')
        continue
    tmp = tmp.rename(columns={title_col: 'title'})
    tmp['body'] = tmp[body_col].fillna('') if body_col else ''
    tmp['label'] = lbl
    tmp['source'] = 'fakenewsnet'

fnn_parts.append(tmp[['title', 'body', 'label', 'source']].dropna(subset=['t
itle'])))
    print(f' {fname} : {len(tmp)} exemples')
except Exception as e:
    print(f' FakeNewsNet {fname} : erreur ({e})')
if fnn_parts:
    dfs.append(pd.concat(fnn_parts))
    print(f' FakeNewsNet total : {sum(len(p) for p in fnn_parts)}
exemples')
else:
    print(' FakeNewsNet : aucun CSV trouvé dans', fnn_dir)

# — FUSION & NETTOYAGE —————
if not dfs:
    print('ERREUR : Aucun dataset chargé ! Vérifier les chemins.')
    sys.exit(1)

print('\nFusion et nettoyage...')
corpus = pd.concat(dfs, ignore_index=True)
print(f'Avant nettoyage : {len(corpus)} exemples')

# Nettoyage texte
corpus['title'] = corpus['title'].apply(clean_text)
corpus['body'] = corpus['body'].apply(clean_text)

# Suppression des titres vides ou trop courts
corpus = corpus[corpus['title'].str.len() > 5]

# Déduplication sur le titre
n_before = len(corpus)
corpus = corpus.drop_duplicates(subset=['title'])
print(f'Doublons supprimés : {n_before - len(corpus)}')
print(f'Corpus final : {len(corpus)} exemples')
print(corpus[['label', 'source']].value_counts().to_string())

# — SPLIT 80/10/10 —————
train_val, test = train_test_split(
    corpus, test_size=0.10, stratify=corpus['label'], random_state=42)

```

```
train, val = train_test_split(
    train_val, test_size=0.111, stratify=train_val['label'],
    random_state=42)

train.to_csv(f'{OUT}/train/train.csv', index=False)
val.to_csv(f'{OUT}/val/val.csv', index=False)
test.to_csv(f'{OUT}/test/test.csv', index=False)

print(f'\nTrain : {len(train)} | Val : {len(val)} | Test : {len(test)}')
print('Prétraitement terminé !')
print(f'Fichiers générés dans {OUT}/')
```

```
# ⚠ ORDRE IMPÉRATIF : toujours lancer depuis la RACINE
$ cd ~/desinformation-pipeline
$ source venv_main/bin/activate

# Étape 1 : Télécharger les datasets
$ bash scripts/download_datasets.sh

# Étape 2 : Prétraiter et fusionner
$ python scripts/preprocess_data.py

# Résultat attendu (environ) :
# WELFake      : 72 134 exemples
# ISOT         : 44 898 exemples
# LIAR         : 12 836 exemples
# FakeNewsNet  : ~23 000 exemples
# Corpus final : ~140 000 - 153 000 exemples (selon le contenu réel des CSV)
# Train : ~122 000 | Val : ~15 000 | Test : ~15 000

# Étape 3 : Vérifier les fichiers générés
$ wc -l data/processed/train/train.csv data/processed/val/val.csv
data/processed/test/test.csv
# Ensuite seulement : lancer l'entraînement
# $ python scripts/train_model.py --epochs 10 --batch_size 16 --lr 2e-5
```

**ÉTAPE
5****Pré-entraînement du Modèle Continual-DistilBERT**

Durée : 4-8h (GPU) / 48h (CPU)

**ASTUCE**

Si vous n'avez pas de GPU local, utiliser Google Colab Pro+ (GRATUIT avec compte Google).

Accès : <https://colab.research.google.com> — Créer un nouveau notebook, choisir Runtime > T4 GPU.

Uploader le script `train_model.py` et le dossier `data/processed/` via Google Drive.

Durée estimée sur T4 GPU (16 Go VRAM) : ~6 heures pour 927 000 exemples.

Durée estimée sur A100 GPU (Colab Pro+) : ~2.5 heures.

5.1 Script d'entraînement principal

**ATTENTION**

CORRECTIONS v2 appliquées dans ce script (erreurs rencontrées en production) :

FIX 1 : `w = torch.tensor([...]).float()` → forcer Float32 pour CrossEntropyLoss

FIX 2 : `lbls = batch['labels'].to(DEVICE).long()` → forcer int64 pour CrossEntropyLoss

FIX 3 : `batch_size` par défaut = 16 (CPU-friendly, évite lenteur excessive)

FIX 4 : `num_workers=0` sur CPU (évite les problèmes de multiprocessing)

Sans ces corrections : `RuntimeError: expected scalar type Float but found Double`

Créer/remplacer le fichier `scripts/train_model.py` avec ce contenu corrigé :

```
python
#!/usr/bin/env python3
# scripts/train_model.py – Pré-entraînement Continual-DistilBERT
# CORRECTIONS v2 : .float() sur les poids, .long() sur les labels,
# batch=16 CPU
# Usage (RACINE du projet, venv_main activé) :
#   python scripts/train_model.py --epochs 10 --batch_size 16 --lr 2e-5

import argparse, os, sys, json
import pandas as pd
import torch
from torch.utils.data import Dataset, DataLoader
from torch.optim import AdamW
from torch.optim.lr_scheduler import LinearLR
from transformers import DistilBertTokenizerFast,
DistilBertForSequenceClassification
from sklearn.metrics import f1_score, roc_auc_score
from tqdm import tqdm

# — Vérification répertoire racine —
if not os.path.exists('scripts/train_model.py'):
    print('ERREUR : Lancer depuis la RACINE : cd ~/desinformation-
```

```

pipeline')
    sys.exit(1)

# — Arguments —————
parser = argparse.ArgumentParser()
parser.add_argument('--epochs', type=int, default=10)
parser.add_argument('--batch_size', type=int, default=16, help='16
recommandé sur CPU')
parser.add_argument('--lr', type=float, default=2e-5)
parser.add_argument('--max_len', type=int, default=128)
parser.add_argument('--model_name', default='distilbert-base-
multilingual-cased')
parser.add_argument('--output_dir', default='models/pretrained')
parser.add_argument('--train_csv',
default='data/processed/train/train.csv')
parser.add_argument('--val_csv', default='data/processed/val/val.csv')
args = parser.parse_args()

DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f'Utilisation de : {DEVICE}')
if DEVICE == 'cuda':
    print(f'GPU : {torch.cuda.get_device_name(0)}')
else:
    print('⚠ CPU détecté → batch_size=16 recommandé. Google Colab pour
accélérer.')

# — Vérification fichiers CSV —————
for path in [args.train_csv, args.val_csv]:
    if not os.path.exists(path):
        print(f'ERREUR : {path} introuvable.')
        print('Exécuter d\'abord : python scripts/preprocess_data.py')
        sys.exit(1)

# — Dataset —————
class NewsDataset(Dataset):
    def __init__(self, csv_path, tokenizer, max_len=128):
        self.df = pd.read_csv(csv_path).fillna('')
        self.df['label'] = self.df['label'].astype(int) # Forcer int
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self): return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        text = f"{str(row['title'])[:200]} [SEP] {str(row['body'])
[:100]}"
        enc = self.tokenizer(text, max_length=self.max_len,
                             padding='max_length', truncation=True,
                             return_tensors='pt')

```

```

    return {
        'input_ids': enc['input_ids'].squeeze(),
        'attention_mask': enc['attention_mask'].squeeze(),
        # ✅ FIX : dtype=torch.long OBLIGATOIRE pour CrossEntropyLoss
        'labels': torch.tensor(int(row['label']), dtype=torch.long)
    }

# — Chargement tokenizer & modèle —————
print(f'Chargement du tokenizer : {args.model_name}')
tokenizer = DistilBertTokenizerFast.from_pretrained(args.model_name)
model      = DistilBertForSequenceClassification.from_pretrained(
    args.model_name, num_labels=2).to(DEVICE)

# — DataLoaders —————
print('Chargement des données...')
train_ds = NewsDataset(args.train_csv, tokenizer, args.max_len)
val_ds   = NewsDataset(args.val_csv,   tokenizer, args.max_len)
# Sur CPU : num_workers=0 évite les erreurs de multiprocessing
n_workers = 0 if DEVICE == 'cpu' else 4
train_dl = DataLoader(train_ds, batch_size=args.batch_size,
    shuffle=True,
                        num_workers=n_workers,
pin_memory=(DEVICE=='cuda'))
val_dl   = DataLoader(val_ds, batch_size=args.batch_size*2,
    shuffle=False,
                        num_workers=n_workers)
print(f'Train : {len(train_ds)} ex. | Val : {len(val_ds)} ex.')

# — Poids de classes —————
lbl_series = pd.read_csv(args.train_csv)['label'].astype(int)
n_fake    = (lbl_series == 1).sum()
n_real    = (lbl_series == 0).sum()
n_total   = len(lbl_series)
print(f'fake={n_fake} ({n_fake/n_total*100:.1f}%) | real={n_real} ({n_real/n_total*100:.1f}%)')
# ✅ FIX : .float() OBLIGATOIRE – CrossEntropyLoss attend Float32
# Sans .float() → RuntimeError: expected scalar type Float but found Double
w          = torch.tensor([n_fake/n_total,
n_real/n_total]).float().to(DEVICE)
loss_fn = torch.nn.CrossEntropyLoss(weight=w)

# — Optimiseur —————
layer_lrs = []
for i, layer in enumerate(model.distilbert.transformer.layer):
    layer_lrs.append({'params': layer.parameters(), 'lr': args.lr * (0.9
** (5-i))})
layer_lrs.append({'params': model.pre_classifier.parameters(), 'lr':
args.lr})
layer_lrs.append({'params': model.classifier.parameters(), 'lr':
args.lr})

```

```

optimizer = AdamW(layer_lrs, weight_decay=0.01)

# — Boucle d'entraînement —————
os.makedirs(args.output_dir, exist_ok=True)
best_f1, history = 0.0, []

for epoch in range(args.epochs):
    scheduler = LinearLR(optimizer, start_factor=0.1, end_factor=1.0,
                          total_iters=max(1, len(train_dl)//5))
    model.train()
    total_loss, preds_all, labels_all = 0.0, [], []
    for batch in tqdm(train_dl, desc=f'Epoch {epoch+1}/{args.epochs}'):
        input_ids = batch['input_ids'].to(DEVICE)
        attn_mask = batch['attention_mask'].to(DEVICE)
        # ✅ FIX : .long() OBLIGATOIRE – CrossEntropyLoss attend int64
        lbls = batch['labels'].to(DEVICE).long()
        optimizer.zero_grad()
        out = model(input_ids=input_ids, attention_mask=attn_mask)
        loss = loss_fn(out.logits, lbls)
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        loss.backward()
        optimizer.step()
        scheduler.step()
        total_loss += loss.item()
        preds_all.extend(out.logits.argmax(-1).cpu().numpy())
        labels_all.extend(lbls.cpu().numpy())
    train_f1 = f1_score(labels_all, preds_all, average='macro')
    print(f'Epoch {epoch+1} | Loss: {total_loss/len(train_dl):.4f} |
Train F1: {train_f1:.4f}')

# — Validation —————
model.eval()
val_preds, val_labels, val_probs = [], [], []
with torch.no_grad():
    for batch in tqdm(val_dl, desc='Validation', leave=False):
        out = model(input_ids=batch['input_ids'].to(DEVICE),
attention_mask=batch['attention_mask'].to(DEVICE))
        probs = torch.softmax(out.logits, dim=-1)
        val_preds.extend(out.logits.argmax(-1).cpu().numpy())
        val_labels.extend(batch['labels'].numpy())
        val_probs.extend(probs[:, 1].cpu().numpy())
val_f1 = f1_score(val_labels, val_preds, average='macro')
try: val_auc = roc_auc_score(val_labels, val_probs)
except: val_auc = 0.0
print(f'Val F1: {val_f1:.4f} | AUC: {val_auc:.4f}')
history.append({'epoch': epoch+1, 'train_f1': round(train_f1,4),
'val_f1': round(val_f1,4)})

if val_f1 > best_f1:

```

```

        best_f1 = val_f1
        model.save_pretrained(args.output_dir)
        tokenizer.save_pretrained(args.output_dir)
        print(f'      ✓ Meilleur modèle sauvegardé ! Val F1 =
{best_f1:.4f}')
        if val_f1 >= 0.93:
            print(f'🛑 F1 >= 0.93 – arrêt anticipé à l\'epoch {epoch+1}')
            break

print(f'Entraînement terminé. Meilleur Val F1 : {best_f1:.4f}')
json.dump(history, open(f'{args.output_dir}/training_history.json', 'w'),
            indent=2)
print('Prochaine étape : python scripts/export_onnx.py')
# — LANCEMENT SUR CPU (ta machine Ubuntu) —————
$ cd ~/desinformation-pipeline && source venv_main/bin/activate
# batch_size=16 recommandé sur CPU (64 = trop lent)
$ python scripts/train_model.py --epochs 10 --batch_size 16 --lr 2e-5

# — LANCEMENT SUR GOOGLE COLAB (recommandé pour aller vite) —
# Runtime > Change runtime type > T4 GPU
# !python scripts/train_model.py --epochs 10 --batch_size 64 --lr 2e-5

# Résultat attendu au début de l'epoch 1 :
# Utilisation de : cpu
# Train : 77495 ex. | Val : 9677 ex.
# fake=XXXX (XX.X%) | real=XXXX (XX.X%)
# Epoch 1/10: ... [00:XX<XX:XX, X.XXit/s] loss=0.XXXX

```

5.2 Export du modèle en ONNX INT8 (optimisation inférence)

```

python
# scripts/export_onnx.py – Conversion PyTorch → ONNX INT8
# Usage : python scripts/export_onnx.py

import torch
from transformers import DistilBertTokenizerFast,
DistilBertForSequenceClassification
from optimum.onnxruntime import ORTModelForSequenceClassification
from optimum.onnxruntime.configuration import AutoQuantizationConfig
import os

MODEL_PATH = 'models/pretrained'
ONNX_PATH = 'models/onnx'
os.makedirs(ONNX_PATH, exist_ok=True)

print('Chargement du modèle PyTorch...')
model = ORTModelForSequenceClassification.from_pretrained(MODEL_PATH,
export=True)

```

```

print('Quantification INT8...')
qconfig = AutoQuantizationConfig.arm64(is_static=False,
per_channel=False)
from optimum.onnxruntime import ORTQuantizer
quantizer = ORTQuantizer.from_pretrained(model)
quantizer.quantize(save_dir=ONNX_PATH, quantization_config=qconfig)

print(f'Modèle ONNX INT8 sauvegardé dans : {ONNX_PATH}')

# Test de l'inférence ONNX
from optimum.onnxruntime import ORTModelForSequenceClassification
from transformers import DistilBertTokenizerFast
import time

tokenizer = DistilBertTokenizerFast.from_pretrained(MODEL_PATH)
ort_model = ORTModelForSequenceClassification.from_pretrained(ONNX_PATH)

test_text = 'BREAKING: Shocking discovery that will change everything!'
inputs = tokenizer(test_text, return_tensors='pt', max_length=128,
truncation=True, padding='max_length')

N_ITER = 100
start = time.time()
for _ in range(N_ITER):
    outputs = ort_model(**inputs)
latency = (time.time() - start) / N_ITER * 1000

import torch.nn.functional as F
probs = F.softmax(torch.tensor(outputs.logits), dim=-1)
label = probs.argmax().item()

print(f'Résultat : {"FAKE" if label == 1 else "RÉEL"} (confiance :
{probs.max().item():.3f})')
print(f'Latence ONNX INT8 : {latency:.2f} ms/article')
# Résultat attendu : ~5-6 ms/article
$ python scripts/export_onnx.py
# Résultat attendu :
# Résultat : FAKE (confiance : 0.921)
# Latence ONNX INT8 : 5.83 ms/article

# Vérifier les fichiers créés
$ ls -lh models/onnx/
# Attendu : model_quantized.onnx (~65 Mo)

```


**ÉTAPE
6****Code Source Complet : Producteur Kafka**
Scraper RSS + GDELT**6.1 Liste des sources RSS (producer/src/rss_sources.py)**

```
python
# producer/src/rss_sources.py

RSS_SOURCES = {
    # — Sources fiables —
    'reliable': [
        {'name': 'AFP', 'url':
'https://www.afp.com/fr/actualites/feeds/feed-afp-com.xml', 'lang':
'fr'},
        {'name': 'Reuters', 'url':
'https://feeds.reuters.com/reuters/topNews', 'lang':
'en'},
        {'name': 'BBC News', 'url':
'https://feeds.bbc.co.uk/news/world/rss.xml', 'lang':
'en'},
        {'name': 'Al Jazeera', 'url':
'https://www.aljazeera.com/xml/rss/all.xml', 'lang': 'en'},
        {'name': 'Le Monde', 'url':
'https://www.lemonde.fr/rss/une.xml', 'lang':
'fr'},
        {'name': 'RFI', 'url': 'https://www.rfi.fr/fr/rss',
'lang': 'fr'},
        {'name': 'AP News', 'url':
'https://rsshub.app/apnews/topics/apf-topnews', 'lang':
'en'},
        {'name': 'Deutsche Welle', 'url': 'https://rss.dw.com/xml/rss-en-
world', 'lang': 'en'},
        {'name': 'France24', 'url':
'https://www.france24.com/fr/rss', 'lang': 'fr'},
        {'name': 'VOA News', 'url': 'https://www.voanews.com/rss/',
'lang': 'en'},
        {'name': 'Jeune Afrique', 'url':
'https://www.jeuneafrique.com/feed/', 'lang': 'fr'},
        {'name': 'Africa News', 'url':
'https://www.africanews.com/feed/rss', 'lang': 'en'},
    ],
    # — Sources à surveiller (potentiellement douteuses) —
    'suspicious': [
        # Configurer via fichier externe pour ne pas exposer les URLs
        # Utiliser la liste de NewsGuard ou de l'IFCN comme référence
    ]
}

# Toutes les sources combinées pour le scraping
ALL_SOURCES = RSS_SOURCES['reliable'] + RSS_SOURCES['suspicious']
```

6.2 Client GDELT (producer/src/gdelt_client.py)

```
python
# producer/src/gdelt_client.py
import httpx, asyncio, json
from datetime import datetime, timedelta

GDELT_API_BASE = 'https://api.gdeltproject.org/api/v2/doc/doc'

async def fetch_gdelt_articles(theme: str = 'FAKE_NEWS,DISINFORMATION',
                               max_records: int = 50) -> list:
    """Récupère les derniers articles GDELT sur la désinformation"""
    params = {
        'query': f'theme:{theme}',
        'mode': 'artlist',
        'maxrecords': max_records,
        'format': 'json',
        'timespan': '15min', # Dernières 15 minutes
        'sort': 'DateDesc',
    }
    async with httpx.AsyncClient(timeout=30) as client:
        resp = await client.get(GDELT_API_BASE, params=params)
        if resp.status_code == 200:
            data = resp.json()
            articles = data.get('articles', [])
            return [{
                'title': a.get('title', ''),
                'url': a.get('url', ''),
                'source': a.get('domain', 'gdelt'),
                'tone': a.get('tone', 0),
                'language': a.get('language', 'English'),
                'timestamp': datetime.utcnow().isoformat(),
                'gdelt_tone': float(a.get('tone', 0)),
            } for a in articles]
    return []
```

6.3 Producteur Kafka principal (producer/src/kafka_producer.py)

```
python
# producer/src/kafka_producer.py – Producteur Kafka complet
import asyncio, feedparser, json, hashlib, logging, os
from datetime import datetime
from confluent_kafka import Producer
from gdelt_client import fetch_gdelt_articles
from rss_sources import ALL_SOURCES
from dotenv import load_dotenv

load_dotenv()
logging.basicConfig(level=logging.INFO, format='%(asctime)s [%'
```

```

(levelname)s] %(message)s')
log = logging.getLogger(__name__)

KAFKA_BROKER = os.getenv('KAFKA_BROKER', 'kafka:9092')
TOPIC        = os.getenv('KAFKA_TOPIC_RAW', 'raw-news-stream')
RSS_INTERVAL = int(os.getenv('RSS_SCRAPED_INTERVAL_SEC', 60))
GDELT_INTERVAL = int(os.getenv('GDELT_QUERY_INTERVAL_SEC', 900))

# — PRODUCTEUR KAFKA —————
producer = Producer({
    'bootstrap.servers': KAFKA_BROKER,
    'client.id': 'rss-scraper-v2',
    'message.max.bytes': 2000000,
    'compression.type': 'gzip',
    'linger.ms': 500,          # Batch messages pour réduire la charge
    'batch.size': 131072,
})

def delivery_callback(err, msg):
    if err: log.error(f'Kafka delivery FAILED: {err}')

seen_ids = set() # Cache des articles déjà envoyés

def send_article(article: dict):
    art_id = hashlib.md5(
        (article.get('url', '') + article.get('title', '')).encode()
    ).hexdigest()
    if art_id in seen_ids: return 0
    seen_ids.add(art_id)
    # Garder seulement les 100 000 derniers IDs en mémoire
    if len(seen_ids) > 100000: seen_ids.pop()
    article['id'] = art_id
    producer.produce(
        topic=TOPIC, key=art_id,
        value=json.dumps(article, ensure_ascii=False).encode('utf-8'),
        callback=delivery_callback
    )
    return 1

async def scrape_rss():
    total = 0
    for src in ALL_SOURCES:
        try:
            feed = feedparser.parse(src['url'])
            for entry in feed.entries[:20]:
                article = {
                    'title': entry.get('title', ''),
                    'body': entry.get('summary', '')[:500],

```

```

        'url': entry.get('link', ''),
        'source': src['name'],
        'source_category': src.get('category', 'reliable'),
        'language': src.get('lang', 'en'),
        'timestamp': datetime.utcnow().isoformat(),
        'gdelt_tone': 0.0,
    }
    total += send_article(article)
except Exception as e:
    log.warning(f'Erreur scraping {src["name"]}: {e}')
producer.flush()
log.info(f'RSS : {total} nouveaux articles envoyés vers Kafka')

async def scrape_gdelt():
    articles = await fetch_gdelt_articles()
    count = sum(send_article(a) for a in articles)
    producer.flush()
    log.info(f'GDELT : {count} articles envoyés')

async def main():
    log.info(f'Producteur Kafka démarré. Broker: {KAFKA_BROKER} | Topic: {TOPIC}')
    rss_counter = 0
    while True:
        await scrape_rss()
        rss_counter += RSS_INTERVAL
        if rss_counter >= GDELT_INTERVAL:
            await scrape_gdelt()
            rss_counter = 0
        await asyncio.sleep(RSS_INTERVAL)

if __name__ == '__main__':
    asyncio.run(main())

```

6.4 Dockerfile du producteur (producer/Dockerfile)

```

dockerfile
# producer/Dockerfile – VERSION CORRIGÉE
# ✅ python:3.12-slim (PAS 3.11-slim)
FROM python:3.12-slim

WORKDIR /app

# Dépendances système pour confluent-kafka (librdkafka)
RUN apt-get update && apt-get install -y --no-install-recommends \
    librdkafka-dev gcc && rm -rf /var/lib/apt/lists/*

# Upgrade pip en premier

```

```

RUN pip install --no-cache-dir --upgrade pip setuptools wheel

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY src/ ./src/

CMD ["python", "src/kafka_producer.py"]

```

6.5 Fichier producer/requirements.txt (contenu exact)

! ATTENTION

! asyncio NE DOIT PAS figurer dans requirements.txt.

asyncio est une bibliothèque standard Python (stdlib) depuis Python 3.4.

Le mettre dans pip peut installer une ancienne version conflictuelle.

Si ton requirements.txt contient 'asyncio==3.4.3', supprimer cette ligne.

```

text
# producer/requirements.txt – CONTENU EXACT (sans asyncio)
# Généré avec : pip freeze > producer/requirements.txt (dans
venv_producer activé)
aiofiles==23.2.1
confluent-kafka==2.5.0
feedparser==6.0.11
gdown==5.2.0
httpx==0.27.0
kaggle==1.6.17
python-dotenv==1.0.1
# asyncio → NE PAS INCLURE (stdlib Python 3.12)

```

ÉTAPE
7Code Source Complet : Spark Structured Streaming
Consumer NLP + Online Learning

7.1 Module NLP — nlp_classifier.py

```
python
# spark-app/src/nlp_classifier.py – Classification ONNX + Online Learning
import os, torch, numpy as np
from transformers import DistilBertTokenizerFast
from onnxruntime import InferenceSession
from torch.optim import AdamW
import torch.nn.functional as F

MODEL_PRETRAINED = os.getenv('MODEL_PRETRAINED_PATH',
                              '/app/models/pretrained')
MODEL_ONNX         = os.getenv('MODEL_ONNX_PATH',
                              '/app/models/onnx/model_quantized.onnx')
ONLINE_LR_BASE     = float(os.getenv('ONLINE_LR_BASE', 1e-5))
RESERVOIR_SIZE     = int(os.getenv('RESERVOIR_BUFFER_SIZE', 5000))

class ContinualDistilBERT:
    """Continual-DistilBERT : inférence ONNX + online learning PyTorch"""

    def __init__(self):
        from transformers import DistilBertForSequenceClassification
        self.tokenizer =
DistilBertTokenizerFast.from_pretrained(MODEL_PRETRAINED)
        # Modèle ONNX pour l'inférence rapide
        self.ort_session = InferenceSession(
            MODEL_ONNX,
            providers=['CPUExecutionProvider']
        )
        # Modèle PyTorch pour les mises à jour online
        self.pt_model =
DistilBertForSequenceClassification.from_pretrained(MODEL_PRETRAINED)
        self.pt_model.train()
        self.optimizer = AdamW(self.pt_model.parameters(),
lr=ONLINE_LR_BASE, weight_decay=0.01)
        # Reservoir buffer pour éviter l'oubli catastrophique
        self.reservoir = []
        self.n_seen = 0
        self.current_lr = ONLINE_LR_BASE
        print('[NLP] Continual-DistilBERT initialisé (ONNX + PyTorch
online)')

    def predict(self, title: str, body: str = '') -> dict:
        """Inférence ONNX (5-6 ms) – NE met PAS à jour le modèle"""
        text = f'{title[:200]} [SEP] {body[:100]}'
        enc = self.tokenizer(text, max_length=128, padding='max_length',
```

```

        truncation=True, return_tensors='np')
logits = self.ort_session.run(None, {
    'input_ids': enc['input_ids'].astype(np.int64),
    'attention_mask': enc['attention_mask'].astype(np.int64)
})[0]
probs = F.softmax(torch.tensor(logits), dim=-1).squeeze()
label = int(probs.argmax().item())
return {'label': label, 'confidence':
float(probs[label]), 'p_fake': float(probs[1])}

def reservoir_update(self, text: str, label: int):
    """Reservoir sampling : garde un échantillon représentatif des
données passées"""
    self.n_seen += 1
    if len(self.reservoir) < RESERVOIR_SIZE:
        self.reservoir.append((text, label))
    else:
        j = np.random.randint(0, self.n_seen)
        if j < RESERVOIR_SIZE:
            self.reservoir[j] = (text, label)

def online_update(self, batch_texts: list, batch_labels: list, lr:
float = None):
    """Mise à jour online sur un batch + replay du reservoir"""
    if lr and lr != self.current_lr:
        for pg in self.optimizer.param_groups: pg['lr'] = lr
        self.current_lr = lr
    # Combiner nouveaux exemples + replay
    replay_size = max(1, len(batch_texts) // 4) # 20% replay
    if len(self.reservoir) >= replay_size:
        idxs = np.random.choice(len(self.reservoir), replay_size,
replace=False)
        replay = [self.reservoir[i] for i in idxs]
        replay_texts = [r[0] for r in replay]
        replay_labels = [r[1] for r in replay]
        all_texts = batch_texts + replay_texts
        all_labels = batch_labels + replay_labels
    else:
        all_texts, all_labels = batch_texts, batch_labels
    # Tokenisation
    enc = self.tokenizer(all_texts, max_length=128,
padding='max_length',
        truncation=True, return_tensors='pt')
    labels_t = torch.tensor(all_labels, dtype=torch.long)
    # Forward + backward
    self.optimizer.zero_grad()
    outputs = self.pt_model(**enc, labels=labels_t)
    torch.nn.utils.clip_grad_norm_(self.pt_model.parameters(), 1.0)
    outputs.loss.backward()
    self.optimizer.step()

```

```

        return float(outputs.loss.item())

    def sync_onnx(self):
        """Exporte le modèle PyTorch mis à jour vers ONNX pour accélérer
        l'inférence"""
        from optimum.onnxruntime import ORTModelForSequenceClassification
        from optimum.onnxruntime.configuration import
        AutoQuantizationConfig
        # Sauvegarder le modèle PyTorch
        checkpoint_dir = os.getenv('MODEL_CHECKPOINT_PATH',
        '/app/models/checkpoints')
        self.pt_model.save_pretrained(checkpoint_dir)
        self.tokenizer.save_pretrained(checkpoint_dir)
        # Re-exporter en ONNX INT8
        ort_model =
        ORTModelForSequenceClassification.from_pretrained(checkpoint_dir,
        export=True)
        from optimum.onnxruntime import ORTQuantizer
        qconfig = AutoQuantizationConfig.arm64(is_static=False,
        per_channel=False)
        quantizer = ORTQuantizer.from_pretrained(ort_model)
        quantizer.quantize(save_dir=os.path.dirname(MODEL_ONNX),
        quantization_config=qconfig)
        # Recharger la session ONNX avec le nouveau modèle
        self.ort_session = InferenceSession(MODEL_ONNX,
        providers=['CPUExecutionProvider'])
        print('[NLP] Modèle ONNX resynchronisé avec les poids online')

```

7.2 Job Spark Streaming principal (spark-app/src/spark_streaming.py)

```

python
# spark-app/src/spark_streaming.py – Job Spark Structured Streaming
principal
import os, json, logging
from pyspark.sql import SparkSession
from pyspark.sql.functions import from_json, col, current_timestamp
from pyspark.sql.types import *
from pymongo import MongoClient
from elasticsearch import Elasticsearch
from confluent_kafka import Producer as KafkaProducer
from nlp_classifier import ContinualDistilBERT
from drift_monitor import DynamicDriftMonitor
from datetime import datetime
from dotenv import load_dotenv

load_dotenv()
log = logging.getLogger(__name__)

# — INITIALISATION SPARK —————
spark = SparkSession.builder \

```



```

.appName('DisinformationPipeline') \
.master('local[*]') \
.master('local[*]') \
.config('spark.streaming.stopGracefullyOnShutdown', 'true') \
.config('spark.sql.shuffle.partitions', '4') \
.config('spark.driver.memory', '4g') \
.config('spark.executor.memory', '4g') \
.getOrCreate()

spark.sparkContext.setLogLevel('WARN')

# — SCHÉMA DES MESSAGES KAFKA —————
NEWS_SCHEMA = StructType([
    StructField('id', StringType()),
    StructField('title', StringType()),
    StructField('body', StringType()),
    StructField('url', StringType()),
    StructField('source', StringType()),
    StructField('source_category', StringType()),
    StructField('language', StringType()),
    StructField('timestamp', StringType()),
    StructField('gdelt_tone', FloatType()),
])

# — LECTURE DEPUIS KAFKA —————
df_raw = spark.readStream \
    .format('kafka') \
    .option('kafka.bootstrap.servers', os.getenv('KAFKA_BROKER',
'kafka:9092')) \
    .option('subscribe', os.getenv('KAFKA_TOPIC_RAW', 'raw-news-stream')) \
    .option('startingOffsets', 'latest') \
    .option('maxOffsetsPerTrigger', '1000') \
    .load()

df_parsed = df_raw \
    .select(from_json(col('value').cast('string'),
NEWS_SCHEMA).alias('d')) \
    .select('d.*') \
    .filter(col('title').isNotNull() & (col('title') != ''))

# — INITIALISATION DES MODULES (dans le driver) —————
nlp_model = ContinualDistilBERT()
drift_monitor = DynamicDriftMonitor()

# Connexions MongoDB et Elasticsearch
mongo_client = MongoClient(os.getenv('MONGO_URI',
'mongodb://mongodb:27017'))
db = mongo_client[os.getenv('MONGO_DB', 'disinformation_db')]
es_client = Elasticsearch(os.getenv('ES_HOST',

```

```

'http://elasticsearch:9200'))

# Producteur Kafka pour les alertes de drift
drift_producer = KafkaProducer({'bootstrap.servers':
os.getenv('KAFKA_BROKER', 'kafka:9092')})

# — TRAITEMENT PAR BATCH —
def process_batch(batch_df, batch_id):
    rows = batch_df.collect()
    if not rows: return
    log.info(f'Batch {batch_id}: {len(rows)} articles')

    batch_texts, batch_labels, batch_docs = [], [], []

    for row in rows:
        # 1. Classification NLP (ONNX - rapide)
        pred = nlp_model.predict(row.title or '', row.body or '')

        # 2. Mise à jour du moniteur de drift
        error_bit = 0 # Supposé correct (supervision partielle)
        drift_result = drift_monitor.update(
            confidence=pred['p_fake'],
            error_bit=error_bit
        )

        # 3. Construction du document enrichi
        doc = {
            'id': row.id, 'title': row.title, 'body': row.body,
            'url': row.url, 'source': row.source,
            'source_category': row.source_category,
            'language': row.language,
            'timestamp': row.timestamp,
            'processed_at': datetime.utcnow().isoformat(),
            'is_fake': pred['label'],
            'confidence': pred['confidence'],
            'p_fake': pred['p_fake'],
            'gdelt_tone': row.gdelt_tone or 0.0,
            'drift_score': drift_result['composite_score'],
            'drift_active': drift_result['drift'],
        }
        batch_docs.append(doc)

        # 4. Accumuler pour l'online learning
        text = f"{row.title} [SEP] {(row.body or '')[:100]}"
        batch_texts.append(text)
        batch_labels.append(pred['label'])
        nlp_model.reservoir_update(text, pred['label'])

    # 5. Online learning sur le batch complet

```

```

    rec_lr = drift_monitor.get_recommended_lr()
    if len(batch_texts) > 0:
        loss = nlp_model.online_update(batch_texts, batch_labels,
lr=rec_lr)
        log.info(f'Online loss: {loss:.4f} | lr: {rec_lr:.2e}')

# 6. Émission d'une alerte si dérive détectée
if drift_monitor.is_drift_active():
    alert = drift_monitor.get_alert_payload()
    drift_producer.produce(
        topic=os.getenv('KAFKA_TOPIC_DRIFT', 'drift-alerts'),
        value=json.dumps(alert).encode('utf-8')
    )
    drift_producer.flush()
    log.warning(f'DRIFT ALERT : score={alert["composite_score"]:.3f}
| détecteurs={alert["signals"]}')

# 7. Stockage MongoDB (bulk insert)
if batch_docs:
    db.articles.insert_many(batch_docs)

# 8. Indexation Elasticsearch (bulk)
from elasticsearch.helpers import bulk
es_docs = [{'_index': 'articles', '_id': d['id'], '_source': d} for d
in batch_docs]
if es_docs:
    bulk(es_client, es_docs, ignore_status=400)

# — LANCEMENT DU STREAMING —————
query = df_parsed.writeStream \
    .foreachBatch(process_batch) \
    .trigger(processingTime='5 seconds') \
    .option('checkpointLocation', os.getenv('SPARK_CHECKPOINT_DIR',
'/tmp/spark-checkpoints')) \
    .start()

log.info('Pipeline Spark Streaming démarré. En attente de données...')
query.awaitTermination()

```

ÉTAPE
8Code Source Complet : Module Concept Drift Dynamique
ADWIN + KSWIN + PageHinkley

8.1 Module drift_monitor.py

```
python
# spark-app/src/drift_monitor.py – Tri-Détecteur de Concept Drift
from river import drift
from dataclasses import dataclass, field
from typing import List, Dict
from datetime import datetime
import os, json

ADWIN_DELTA = float(os.getenv('ADWIN_DELTA', 0.002))
KSWIN_WINDOW = int(os.getenv('KSWIN_WINDOW_SIZE', 100))
KSWIN_ALPHA = float(os.getenv('KSWIN_ALPHA', 0.005))
DRIFT_THRESH = float(os.getenv('DRIFT_COMPOSITE_THRESHOLD', 0.5))
CONFIRM_THRESH = float(os.getenv('DRIFT_CONFIRMED_THRESHOLD', 0.8))
LR_BASE = float(os.getenv('ONLINE_LR_BASE', 1e-5))
LR_DRIFT = float(os.getenv('ONLINE_LR_DRIFT', 5e-5))

class DynamicDriftMonitor:
    """
    Tri-Détecteur de Concept Drift :
    - ADWIN (poids 0.45) : surveillance de la moyenne du score de
    confiance
    - KSWIN (poids 0.35) : test statistique KS sur la distribution du
    score
    - PageHinkley (poids 0.20) : somme cumulée, détection de dérives
    graduelles (River 0.21.2)
    """
    WEIGHTS = {'ADWIN': 0.45, 'KSWIN': 0.35, 'PageHinkley': 0.20}

    def __init__(self):
        self.adwin = drift.ADWIN(delta=ADWIN_DELTA)
        self.kswin = drift.KSWIN(window_size=KSWIN_WINDOW,
                                stat_size=KSWIN_WINDOW//3,
                                alpha=KSWIN_ALPHA)
        self.page_hinkley = drift.PageHinkley(min_instances=30,
                                              delta=0.005, threshold=50.0, alpha=0.9999)
        self.composite_score = 0.0
        self.drift_active = False
        self.drift_confirmed = False
        self.messages_total = 0
        self.messages_since_drift = 0
        self.drift_events = []
        self.confidence_history = []
        print(f'[DRIFT] Monitor initialisé | ADWIN δ={ADWIN_DELTA} |
        KSWIN α={KSWIN_ALPHA}')
```

```

def update(self, confidence: float, error_bit: int = 0) -> Dict:
    self.messages_total += 1
    self.confidence_history.append(confidence)
    if len(self.confidence_history) > 1000:
        self.confidence_history.pop(0)

    # — Mise à jour des 3 détecteurs —————
    self.adwin.update(confidence)
    self.kswin.update(confidence)
    self.page_hinkley.update(confidence)

    # — Score composite pondéré —————
    signals = {
        'ADWIN': float(self.adwin.drift_detected),
        'KSWIN': float(self.kswin.drift_detected),
        'PageHinkley': float(self.page_hinkley.drift_detected),
    }
    self.composite_score = sum(self.WEIGHTS[k]*v for k,v in
signals.items())

    # — Logique de décision —————
    if self.composite_score >= DRIFT_THRESH:
        if not self.drift_active:
            # Nouveau drift détecté
            self.drift_events.append({
                'timestamp': datetime.utcnow().isoformat(),
                'message_index': self.messages_total,
                'score': self.composite_score,
                'signals': signals,
            })
            self.drift_active = True
            self.drift_confirmed = self.composite_score >= CONFIRM_THRESH
            self.messages_since_drift = 0
        else:
            self.messages_since_drift += 1
            if self.messages_since_drift > 1000:
                # Cooldown : la dérive est passée
                self.drift_active = False
                self.drift_confirmed = False

    return {
        'drift': self.drift_active,
        'composite_score': round(self.composite_score, 4),
        'signals': signals,
        'recommended_lr': LR_DRIFT if self.drift_active else LR_BASE,
    }

def get_recommended_lr(self) -> float:

```

```
        return LR_DRIFT if self.drift_active else LR_BASE

    def is_drift_active(self) -> bool:
        return self.drift_active

    def get_alert_payload(self) -> dict:
        conf_window = self.confidence_history[-100:]
        return {
            'timestamp': datetime.utcnow().isoformat(),
            'composite_score': round(self.composite_score, 4),
            'signals': {
                'ADWIN': bool(self.adwin.drift_detected),
                'KSWIN': bool(self.kswin.drift_detected),
                'PageHinkley': bool(self.eddm.drift_detected),
            },
            'drift_confirmed': self.drift_confirmed,
            'recommended_lr': self.get_recommended_lr(),
            'messages_total': self.messages_total,
            'confidence_mean_last100':
round(sum(conf_window)/len(conf_window),4) if conf_window else 0.0,
            'total_drift_events': len(self.drift_events),
        }

    def get_stats(self) -> dict:
        """Statistiques pour le dashboard Grafana"""
        return {
            'messages_total': self.messages_total,
            'composite_score': round(self.composite_score, 4),
            'drift_active': self.drift_active,
            'drift_confirmed': self.drift_confirmed,
            'total_drift_events': len(self.drift_events),
            'current_lr': self.get_recommended_lr(),
        }
```

ÉTAPE
9**Code Source Complet : API REST FastAPI**

Exposition des données en temps réel

9.1 Application FastAPI principale (api/src/main.py)

```
python
# api/src/main.py – API REST FastAPI
from fastapi import FastAPI, HTTPException, Query
from fastapi.middleware.cors import CORSMiddleware
from pymongo import MongoClient
from elasticsearch import Elasticsearch
from pydantic import BaseModel
from typing import List, Optional
from datetime import datetime, timedelta
import os
from dotenv import load_dotenv

load_dotenv()

app = FastAPI(
    title='Disinformation Monitor API',
    description='API temps réel pour le monitoring de la désinformation',
    version='2.0.0'
)

app.add_middleware(CORSMiddleware,
    allow_origins=['*'], allow_credentials=True,
    allow_methods=['*'], allow_headers=['*'])

# — CONNEXIONS —
mongo = MongoClient(os.getenv('MONGO_URI', 'mongodb://mongodb:27017'))
db = mongo[os.getenv('MONGO_DB', 'disinformation_db')]
es = Elasticsearch(os.getenv('ES_HOST', 'http://elasticsearch:9200'))

# — ENDPOINTS —
@app.get('/health')
def health():
    return {'status': 'ok', 'timestamp': datetime.utcnow().isoformat()}

@app.get('/api/v1/stats')
def get_stats():
    """Statistiques globales en temps réel"""
    total = db.articles.count_documents({})
    n_fake = db.articles.count_documents({'is_fake': 1})
    n_real = db.articles.count_documents({'is_fake': 0})
    n_drift = db.drift_events.count_documents({})
    last_1h = db.articles.count_documents({
```

```

        'processed_at': {'$gte': (datetime.utcnow()-
timedelta(hours=1)).isoformat()}
    })
    return {
        'total_articles': total,
        'fake_articles': n_fake,
        'real_articles': n_real,
        'fake_rate': round(n_fake/total*100,2) if total>0 else 0,
        'drift_events': n_drift,
        'articles_last_hour': last_1h,
        'timestamp': datetime.utcnow().isoformat()
    }

@app.get('/api/v1/articles/recent')
def get_recent_articles(
    limit: int = Query(50, le=200),
    fake_only: bool = False
):
    """Derniers articles classifiés"""
    query = {'is_fake': 1} if fake_only else {}
    arts = list(db.articles.find(query, {'_id':0}).sort('processed_at', -
1).limit(limit))
    return {'articles': arts, 'count': len(arts)}

@app.get('/api/v1/drift/events')
def get_drift_events(limit: int = Query(20, le=100)):
    """Historique des événements de Concept Drift"""
    events = list(db.drift_events.find({},{'_id':0}).sort('timestamp', -
1).limit(limit))
    return {'events': events, 'count': len(events)}

@app.get('/api/v1/search')
def search_articles(q: str = Query(..., min_length=2), limit: int = 20):
    """Recherche full-text via Elasticsearch"""
    res = es.search(index='articles', body={
        'query': {'multi_match': {'query': q, 'fields':
['title^3','body']}},
        'size': limit
    })
    hits = [h['_source'] for h in res['hits']['hits']]
    return {'results': hits, 'total': res['hits']['total']['value']}

@app.get('/api/v1/virality')
def get_virality_trend(hours: int = Query(24, le=168)):
    """Évolution du taux de fakes sur les dernières N heures"""
    pipeline = [
        {'$match': {'processed_at': {'$gte': (datetime.utcnow()-
timedelta(hours=hours)).isoformat()}}},
        {'$group': {
            '_id': {'$substr': ['$processed_at', 0, 13]}, # Par heure

```



```

        'total': {'$sum': 1},
        'fakes': {'$sum': '$is_fake'},
        'avg_confidence': {'$avg': '$confidence'}
    }},
    {'$sort': {'_id': 1}}
]
data = list(db.articles.aggregate(pipeline))
return {'trend': data}

```

9.2 Dockerfile API (api/Dockerfile)

```

dockerfile
# api/Dockerfile — VERSION CORRIGÉE
#  python:3.12-slim (PAS 3.11-slim)
FROM python:3.12-slim
WORKDIR /app

RUN pip install --no-cache-dir --upgrade pip setuptools wheel

# Versions épinglées et compatibles Python 3.12
RUN pip install --no-cache-dir \
    fastapi==0.113.0 \
    uvicorn==0.30.6 \
    pymongo==4.9.1 \
    elasticsearch==8.15.0 \
    python-dotenv==1.0.1 \
    pydantic==2.9.2

COPY src/ ./src/

CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8000",
    "--workers", "2"]

```

**ÉTAPE
10****Configuration Docker Compose Complète**

Orchestration de tous les services

Créer le fichier docker-compose.yml à la racine du projet :

```

yaml
# docker-compose.yml – Pipeline Big Data Désinformation
# Version : 2025 – KOMOSSI Sosso – UCAO UUT Master BIG DATA IA
# Usage : docker compose up -d

services:

# =====
# ZOOKEEPER – Coordinateur Kafka
# =====
zookeeper:
  image: confluentinc/cp-zookeeper:7.6.0
  container_name: zookeeper
  restart: unless-stopped
  environment:
    ZOOKEEPER_CLIENT_PORT: 2181
    ZOOKEEPER_TICK_TIME: 2000
    ZOOKEEPER_SYNC_LIMIT: 2
  volumes:
    - zookeeper_data:/var/lib/zookeeper/data
    - zookeeper_log:/var/lib/zookeeper/log
  healthcheck:
    test: ['CMD', 'nc', '-z', 'localhost', '2181']
    interval: 10s
    timeout: 5s
    retries: 5

# =====
# KAFKA BROKER
# =====
kafka:
  image: confluentinc/cp-kafka:7.6.0
  container_name: kafka
  restart: unless-stopped
  depends_on:
    zookeeper:
      condition: service_healthy
  ports:
    - '9092:9092'
  environment:
    KAFKA_BROKER_ID: 1
    KAFKA_ZOOKEEPER_CONNECT: 'zookeeper:2181'
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:

```

```

PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
  KAFKA_ADVERTISED_LISTENERS:
PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092
  KAFKA_LISTENERS:
PLAINTEXT://0.0.0.0:29092,PLAINTEXT_HOST://0.0.0.0:9092
  KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
  KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
  KAFKA_AUTO_CREATE_TOPICS_ENABLE: 'true'
  KAFKA_NUM_PARTITIONS: 6
  KAFKA_LOG_RETENTION_HOURS: 24
  KAFKA_MESSAGE_MAX_BYTES: 2000000
volumes:
  - kafka_data:/var/lib/kafka/data
healthcheck:
  test: ['CMD', 'kafka-topics', '--list', '--bootstrap-server',
'localhost:29092']
  interval: 15s
  timeout: 10s
  retries: 5

# =====
# KAFKA UI (Kafdrop) – Monitoring Kafka
# =====
kafdrop:
  image: obsidiandynamics/kafdrop:latest
  container_name: kafdrop
  restart: unless-stopped
  depends_on: [kafka]
  ports:
    - '9000:9000'
  environment:
    KAFKA_BROKERCONNECT: 'kafka:29092'
    JVM_OPTS: '-Xms32M -Xmx64M'

# =====
# SPARK MASTER
# =====
spark-master:
  image: bitnami/spark:3.5
  container_name: spark-master
  restart: unless-stopped
  environment:
    SPARK_MODE: master
    SPARK_RPC_AUTHENTICATION_ENABLED: 'no'
    SPARK_RPC_ENCRYPTION_ENABLED: 'no'
    SPARK_LOCAL_STORAGE_ENCRYPTION_ENABLED: 'no'
    SPARK_SSL_ENABLED: 'no'
    SPARK_MASTER_OPTS: '-Dspark.deploy.defaultCores=2'
  ports:
    - '8080:8080'

```

```
- '7077:7077'
volumes:
  - spark_logs:/opt/bitnami/spark/logs

# =====
# SPARK WORKER × 2
# =====

spark-worker-1:
  image: bitnami/spark:3.5
  container_name: spark-worker-1
  restart: unless-stopped
  depends_on: [spark-master]
  environment:
    SPARK_MODE: worker
    SPARK_MASTER_URL: spark://spark-master:7077
    SPARK_WORKER_MEMORY: 4G
    SPARK_WORKER_CORES: 2

spark-worker-2:
  image: bitnami/spark:3.5
  container_name: spark-worker-2
  restart: unless-stopped
  depends_on: [spark-master]
  environment:
    SPARK_MODE: worker
    SPARK_MASTER_URL: spark://spark-master:7077
    SPARK_WORKER_MEMORY: 4G
    SPARK_WORKER_CORES: 2

# =====
# MONGODB
# =====

mongodb:
  image: mongo:7.0
  container_name: mongodb
  restart: unless-stopped
  ports:
    - '27017:27017'
  volumes:
    - mongodb_data:/data/db
    - ./config/mongodb/init.js:/docker-entrypoint-initdb.d/init.js:ro
  healthcheck:
    test: ['CMD', 'mongosh', '--eval', 'db.runCommand("ping").ok']
    interval: 10s
    timeout: 5s
    retries: 5

# =====
# ELASTICSEARCH
```

```

# =====
elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:8.14.0
  container_name: elasticsearch
  restart: unless-stopped
  environment:
    - discovery.type=single-node
    - xpack.security.enabled=false
    - ES_JAVA_OPTS=-Xms2g -Xmx2g
    - cluster.name=disinformation-cluster
    - bootstrap.memory_lock=true
  ulimits:
    memlock:
      soft: -1
      hard: -1
  ports:
    - '9200:9200'
  volumes:
    - elasticsearch_data:/usr/share/elasticsearch/data
  healthcheck:
    test: ['CMD-SHELL', 'curl -sf http://localhost:9200/_cat/health |
grep -qE "green|yellow"']
    interval: 20s
    timeout: 10s
    retries: 10

# =====
# GRAFANA
# =====
grafana:
  image: grafana/grafana:10.4.0
  container_name: grafana
  restart: unless-stopped
  depends_on: [elasticsearch, mongodb]
  ports:
    - '3000:3000'
  environment:
    GF_SECURITY_ADMIN_USER: ${GRAFANA_ADMIN_USER:-admin}
    GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_ADMIN_PASSWORD:-admin2025}
    GF_INSTALL_PLUGINS: grafana-mongodb-datasource,grafana-
elasticsearch-datasource
    GF_USERS_ALLOW_SIGN_UP: 'false'
  volumes:
    - grafana_data:/var/lib/grafana
    - ./config/grafana/provisioning:/etc/grafana/provisioning
    - ./config/grafana/dashboards:/var/lib/grafana/dashboards

# =====
# RSS PRODUCER
# =====

```

```
rss-producer:
  build:
    context: ./producer
    dockerfile: Dockerfile
  container_name: rss-producer
  restart: unless-stopped
  depends_on:
    kafka:
      condition: service_healthy
  env_file: .env
  environment:
    KAFKA_BROKER: kafka:29092
  volumes:
    - ./logs:/app/logs

# =====
# SPARK APPLICATION (Streaming NLP + Concept Drift)
# =====

spark-app:
  build:
    context: ./spark-app
    dockerfile: Dockerfile
  container_name: spark-app
  restart: unless-stopped
  depends_on:
    kafka:
      condition: service_healthy
    mongodb:
      condition: service_healthy
    elasticsearch:
      condition: service_healthy
  env_file: .env
  environment:
    KAFKA_BROKER: kafka:29092
    MONGO_URI: mongodb://mongodb:27017
    ES_HOST: http://elasticsearch:9200
  volumes:
    - ./models:/app/models          # Partage des modèles entraînés
    - ./logs:/app/logs
    - spark_checkpoints:/tmp/spark-checkpoints
  deploy:
    resources:
      limits:
        memory: 8G

# =====
# API FASTAPI
# =====

api:
```

```

build:
  context: ./api
  dockerfile: Dockerfile
container_name: api
restart: unless-stopped
depends_on: [mongodb, elasticsearch]
ports:
  - '8000:8000'
env_file: .env
environment:
  MONGO_URI: mongodb://mongodb:27017
  ES_HOST: http://elasticsearch:9200

# =====
# VOLUMES PERSISTANTS
# =====
volumes:
  zookeeper_data:
  zookeeper_log:
  kafka_data:
  mongodb_data:
  elasticsearch_data:
  grafana_data:
  spark_logs:
  spark_checkpoints:

```

10.2 Dockerfile Spark App (spark-app/Dockerfile)

ERREUR FRÉQUENTE

ERREUR RENCONTRÉE : curl: (23) Failed writing body — exit code 23

CAUSE : L'image apache/spark:3.5.1 utilise Python 3.8 ET n'a pas de dossier /opt/bitnami/spark/jars/. Le chemin est /opt/spark/jars/ dans apache/spark.

SOLUTION : Utiliser UNIQUEMENT bitnami/spark:3.5 (Python 3.12, chemin correct).

Ne jamais substituer apache/spark:X.X.X à la place de bitnami/spark:3.5.

```

dockerfile
# spark-app/Dockerfile — VERSION CORRIGÉE
# ⚠ Utiliser UNIQUEMENT bitnami/spark:3.5 (PAS apache/spark:3.5.1)
# bitnami/spark:3.5 → Python 3.12 + /opt/bitnami/spark/jars/ déjà présent
# apache/spark:3.5.1 → Python 3.8 + /opt/spark/jars/ → INCOMPATIBLE
FROM bitnami/spark:3.5
USER root

# Étape 1 : Mise à jour pip + packaging (évite les conflits de
dépendances)
RUN pip install --no-cache-dir --upgrade pip setuptools wheel && \
    pip install --no-cache-dir 'packaging>=23.2'

```

```

# Étape 2 : PyTorch CPU (séparé pour bénéficier du cache Docker)
RUN pip install --no-cache-dir \
    torch==2.4.0 \
    --index-url https://download.pytorch.org/whl/cpu

# Étape 3 : Toutes les autres dépendances ML/NLP
RUN pip install --no-cache-dir \
    transformers==4.44.0 \
    river==0.21.2 # ADWIN, KSWIN, PageHinkley # ADWIN, KSWIN,
    PageHinkley \
    onnxruntime==1.19.0 \
    optimum==1.21.0 \
    pymongo==4.9.1 \
    elasticsearch==8.15.0 \
    confluent-kafka==2.5.0 \
    pandas==2.2.2 \
    numpy==1.26.4 \
    vaderSentiment==3.3.2 \
    python-dotenv==1.0.1

# Étape 4 : JARs Kafka pour Spark
# ✅ mkdir -p en sécurité – le dossier existe déjà dans bitnami/spark:3.5
# ⚠️ NE PAS changer le chemin /opt/bitnami/spark/jars/
# Ce chemin n'existe PAS dans apache/spark → erreur exit code 23
RUN mkdir -p /opt/bitnami/spark/jars && \
    curl -fsSL https://repo1.maven.org/maven2/org/apache/spark/spark-sql-
    kafka-0-10_2.12/3.5.1/spark-sql-kafka-0-10_2.12-3.5.1.jar \
    -o /opt/bitnami/spark/jars/spark-sql-kafka.jar && \
    curl -fsSL https://repo1.maven.org/maven2/org/apache/kafka/kafka-
    clients/3.5.1/kafka-clients-3.5.1.jar \
    -o /opt/bitnami/spark/jars/kafka-clients.jar && \
    curl -fsSL https://repo1.maven.org/maven2/org/apache/spark/spark-
    token-provider-kafka-0-10_2.12/3.5.1/spark-token-provider-kafka-0-
    10_2.12-3.5.1.jar \
    -o /opt/bitnami/spark/jars/spark-token-provider-kafka.jar && \
    curl -fsSL https://repo1.maven.org/maven2/org/apache/commons/commons-
    pool2/2.11.1/commons-pool2-2.11.1.jar \
    -o /opt/bitnami/spark/jars/commons-pool2.jar
# Note : curl -fsSL = fail silently + follow redirects. Échoue proprement
si URL invalide.

WORKDIR /app
COPY src/ ./src/

CMD ["python", "src/spark_streaming.py"]

```


**ÉTAPE
11****Configuration MongoDB & Elasticsearch**

Schémas, Index et Initialisation

11.1 Script d'initialisation MongoDB (config/mongodb/init.js)

```

javascript
// config/mongodb/init.js
// Exécuté automatiquement au démarrage du container MongoDB

db = db.getSiblingDB('disinformation_db');

// — COLLECTION ARTICLES —————
db.createCollection('articles', {
  validator: {
    $jsonSchema: {
      bsonType: 'object',
      required: ['id', 'title', 'is_fake', 'processed_at'],
      properties: {
        id: { bsonType: 'string' },
        title: { bsonType: 'string' },
        body: { bsonType: 'string' },
        url: { bsonType: 'string' },
        source: { bsonType: 'string' },
        is_fake: { bsonType: 'int', minimum: 0, maximum: 1 },
        confidence: { bsonType: 'double', minimum: 0, maximum: 1 },
        p_fake: { bsonType: 'double', minimum: 0, maximum: 1 },
        drift_score: { bsonType: 'double' },
        processed_at: { bsonType: 'string' },
      }
    }
  },
  validationAction: 'warn' // Avertir sans bloquer
});

// — INDEX POUR LES REQUÊTES FRÉQUENTES —————
db.articles.createIndex({ 'processed_at': -1 });
db.articles.createIndex({ 'is_fake': 1, 'processed_at': -1 });
db.articles.createIndex({ 'source': 1 });
db.articles.createIndex({ 'drift_score': -1 });
db.articles.createIndex({ 'id': 1 }, { unique: true });

// — COLLECTION DRIFT EVENTS —————
db.createCollection('drift_events');
db.drift_events.createIndex({ 'timestamp': -1 });

print('MongoDB initialisé avec succès !');
print('Collections : articles, drift_events');
print('Index créés sur : processed_at, is_fake, source, drift_score,

```

```
id');
```

11.2 Mapping Elasticsearch (config/elasticsearch/articles_mapping.json)

```
json
{
  "mappings": {
    "properties": {
      "id": { "type": "keyword" },
      "title": { "type": "text", "analyzer": "standard",
        "fields": { "keyword": { "type": "keyword" } } },
      "body": { "type": "text", "analyzer": "standard" },
      "source": { "type": "keyword" },
      "language": { "type": "keyword" },
      "is_fake": { "type": "integer" },
      "confidence": { "type": "float" },
      "p_fake": { "type": "float" },
      "drift_score": { "type": "float" },
      "drift_active": { "type": "boolean" },
      "gdelt_tone": { "type": "float" },
      "processed_at": { "type": "date", "format":
"strict_date_optional_time" },
      "timestamp": { "type": "date", "format":
"strict_date_optional_time" }
    }
  },
  "settings": {
    "number_of_shards": 1,
    "number_of_replicas": 0,
    "refresh_interval": "1s"
  }
}
```

11.3 Script de création des index Elasticsearch

```
# Attendre qu'Elasticsearch soit démarré (status green ou yellow)
$ until curl -s http://localhost:9200/_cluster/health | grep -E
'"status":"green"|"status":"yellow"'; do
  echo 'Attente Elasticsearch...' && sleep 5
done

# Créer l'index articles avec le mapping
$ curl -X PUT 'http://localhost:9200/articles' \
  -H 'Content-Type: application/json' \
  -d @config/elasticsearch/articles_mapping.json

# Créer l'index drift-events
$ curl -X PUT 'http://localhost:9200/drift-events' \
  -H 'Content-Type: application/json' \
```

```
-d '{"mappings":{"properties":{"timestamp":  
{"type":"date"},"composite_score":{"type":"float"},"drift_confirmed":  
{"type":"boolean"}}}}'
```

```
# Vérifier
```

```
$ curl -s http://localhost:9200/_cat/indices?v
```

```
# Attendu : index 'articles' et 'drift-events' avec status green
```

ÉTAPE 12 Configuration Grafana Dashboard

7 panneaux temps réel

12.1 Provisioning des datasources (config/grafana/provisioning/datasources/datasources.yaml)

```
yaml
# config/grafana/provisioning/datasources/datasources.yaml
apiVersion: 1

datasources:
  # — Elasticsearch —————
  - name: Elasticsearch
    type: elasticsearch
    access: proxy
    url: http://elasticsearch:9200
    database: articles
    isDefault: true
    jsonData:
      timeField: processed_at
      esVersion: '8.14.0'
      interval: Daily
      maxConcurrentShardRequests: 5
      logMessageField: title
      logLevelField: is_fake

  # — Elasticsearch Drift Events —————
  - name: Elasticsearch-Drift
    type: elasticsearch
    access: proxy
    url: http://elasticsearch:9200
    database: drift-events
    jsonData:
      timeField: timestamp
      esVersion: '8.14.0'

  # — API FastAPI (via Infinity plugin ou JSON datasource) —
  - name: DisinformationAPI
    type: marcusolsson-json-datasource
    access: proxy
    url: http://api:8000
    jsonData:
      queryParams: ''
```

12.2 Dashboard JSON principal (résumé des 7 panneaux)

Créer le fichier `config/grafana/dashboards/main_dashboard.json` avec le contenu suivant. Ce fichier définit le tableau de bord complet qui sera automatiquement chargé au démarrage de Grafana :

ASTUCE

Le fichier JSON complet du dashboard fait ~500 lignes. Il est disponible dans le dépôt GitHub du projet.

Alternativement, créer les panneaux manuellement dans Grafana (<http://localhost:3000>) après démarrage.

Identifiants par défaut : admin / admin2025

Aller dans : Dashboards > New > Import > Coller le JSON ou Upload JSON file.

12.3 Création manuelle des 7 panneaux Grafana

Voici la configuration de chaque panneau à créer dans Grafana :

N°	Nom du panneau	Type	Source / Requête	Seuil / Alerte
P1	Taux de Fakes (%)	Gauge	ES: <code>count(is_fake=1)/count()*100</code> — 5min	> 35 % → Warning, > 50 % → Critical
P2	Flux d'articles (count/h)	Time Series	ES: <code>count() groupBy 5min intervals</code>	—
P3	Score Composite Drift	Time Series	ES: <code>avg(drift_score) groupBy 30s</code>	> 0.5 → Zone Alerte (orange), > 0.8 → Zone Dérive (rouge)
P4	Alertes de Dérive	Table	ES index drift-events, colonnes: <code>timestamp,score,signals</code>	—
P5	Learning Rate Online	Stat	API <code>/api/v1/stats</code> → <code>current_lr</code>	—
P6	Top Sources Fake	Pie Chart	ES: terms aggregation sur source, filter <code>is_fake=1</code>	—
P7	F1-Score Prequential	Time Series	API <code>/api/v1/stats</code> → <code>f1_prequential</code>	< 0.85 → Warning

12.4 Configuration du provisioning des dashboards

```
yaml
# config/grafana/provisioning/dashboards/dashboards.yaml
apiVersion: 1

providers:
  - name: 'Default'
    orgId: 1
    folder: 'Disinformation Monitor'
```

```
type: file
disableDeletion: false
updateIntervalSeconds: 30
allowUiUpdates: true
options:
  path: /var/lib/grafana/dashboards
  foldersFromFilesStructure: true
```

ÉTAPE 13 Lancement & Démarrage du Pipeline

Du zéro au pipeline opérationnel

13.1 Ordre de démarrage recommandé

INFO

Docker Compose gère l'ordre de démarrage via les 'depends_on' et healthchecks.
 Le démarrage complet prend 2-4 minutes selon la puissance de la machine.
 Suivre les logs avec : `docker compose logs -f` pour surveiller le démarrage.

13.2 Commande de démarrage complet

```
$ cd ~/desinformation-pipeline

# — VÉRIFICATION PRÉ-LANCEMENT —————
# 1. Vérifier que le modèle est bien entraîné et présent
$ ls -lh models/pretrained/
# Attendu : config.json, pytorch_model.bin (ou model.safetensors),
vocab.txt

$ ls -lh models/onnx/
# Attendu : model_quantized.onnx (~65 Mo)

# 2. Vérifier le .env
$ cat .env | head -20

# — DÉMARRAGE —————
# Option A : Démarrage en arrière-plan (recommandé pour la production)
$ docker compose up -d

# Option B : Démarrage avec logs visibles (recommandé pour
debug/soutenance)
$ docker compose up

# — SUIVI DU DÉMARRAGE —————
# Voir les logs de tous les services
$ docker compose logs -f

# Voir les logs d'un service spécifique
$ docker compose logs -f kafka
$ docker compose logs -f spark-app
$ docker compose logs -f rss-producer

# — VÉRIFICATION QUE TOUT EST UP —————
$ docker compose ps
# Attendu : tous les services à l'état 'running' ou 'healthy'
```

13.3 Création des topics Kafka (si non auto-crés)

```
# Attendre que Kafka soit prêt
$ docker compose exec kafka kafka-topics --list --bootstrap-server localhost:9092

# Créer les topics manuellement si nécessaire
$ docker compose exec kafka kafka-topics \
  --create --bootstrap-server localhost:9092 \
  --topic raw-news-stream \
  --partitions 6 \
  --replication-factor 1

$ docker compose exec kafka kafka-topics \
  --create --bootstrap-server localhost:9092 \
  --topic classified-news \
  --partitions 6 \
  --replication-factor 1

$ docker compose exec kafka kafka-topics \
  --create --bootstrap-server localhost:9092 \
  --topic drift-alerts \
  --partitions 1 \
  --replication-factor 1

# Vérifier les topics créés
$ docker compose exec kafka kafka-topics --list --bootstrap-server localhost:9092
# Attendu : drift-alerts, classified-news, raw-news-stream
```

13.4 Démarrage des services Elasticsearch et MongoDB

```
# Créer les index Elasticsearch (après démarrage)
$ curl -X PUT 'http://localhost:9200/articles' \
  -H 'Content-Type: application/json' \
  -d @config/elasticsearch/articles_mapping.json

# Vérifier Elasticsearch
$ curl http://localhost:9200/_cluster/health?pretty
# Attendu : 'status': 'green' ou 'yellow'

# Vérifier MongoDB
$ docker compose exec mongodb mongosh --eval 'db.runCommand({ping:1})'
# Attendu : { ok: 1 }
```


ÉTAPE 14 Vérification et Tests de Bon Fonctionnement

Checklist complète

14.1 Checklist de vérification — Services de base

Service	URL / Commande de test	Résultat attendu	Statut
Kafka Broker	<code>docker compose exec kafka kafka-topics --list --bootstrap-server localhost:9092</code>	Liste des topics (raw-news-stream...)	✓ / ✗
Kafka UI (Kafdrop)	<code>http://localhost:9000</code>	Interface web avec liste des topics	✓ / ✗
Spark Master UI	<code>http://localhost:8080</code>	Interface Spark avec 2 workers actifs	✓ / ✗
MongoDB	<code>docker compose exec mongodb mongosh --eval 'db.stats()'</code>	Collections articles, drift_events	✓ / ✗
Elasticsearch	<code>curl http://localhost:9200/_cat/indices?v</code>	Index articles et drift-events listés	✓ / ✗
Grafana	<code>http://localhost:3000</code>	Page de login (admin/admin2025)	✓ / ✗
API FastAPI	<code>http://localhost:8000/health</code>	<code>{'status': 'ok', 'timestamp': '...'}</code>	✓ / ✗
API Docs	<code>http://localhost:8000/docs</code>	Documentation Swagger interactive	✓ / ✗

14.2 Test du flux de données de bout en bout

```
# — TEST 1 : Vérifier que des messages arrivent dans Kafka —
$ docker compose exec kafka kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic raw-news-stream \
  --from-beginning \
  --max-messages 3
# Attendu : 3 messages JSON avec titre, source, timestamp

# — TEST 2 : Vérifier que Spark consomme les messages —
$ docker compose logs spark-app | tail -30
# Attendu : 'Batch N: X articles' toutes les 2 secondes
# Attendu : 'Online loss: 0.XXX | lr: 1e-05'

# — TEST 3 : Vérifier les données dans MongoDB —
$ docker compose exec mongodb mongosh disinformation_db \
  --eval 'db.articles.countDocuments()'
# Attendu : nombre croissant d'articles (> 0 après 2-3 minutes)
```

```
$ docker compose exec mongodb mongosh disinformation_db \
  --eval 'db.articles.findOne({}, {title:1, is_fake:1, confidence:1,
  _id:0})'
# Attendu : document avec titre, is_fake (0 ou 1), confidence (0.0-1.0)

# — TEST 4 : Vérifier l'API REST —————
$ curl -s http://localhost:8000/api/v1/stats | python3 -m json.tool
# Attendu : total_articles > 0, fake_rate (%), articles_last_hour

$ curl -s 'http://localhost:8000/api/v1/articles/recent?limit=3' |
python3 -m json.tool
# Attendu : liste de 3 articles avec champs complets

# — TEST 5 : Test de recherche Elasticsearch —————
$ curl -s 'http://localhost:8000/api/v1/articles/search?q=fake+news' |
python3 -m json.tool
# Attendu : résultats avec score de pertinence
```

14.3 Test de la détection du Concept Drift

```
# Simuler une dérive abrupte en injectant des messages fake dans Kafka
$ python3 - << 'EOF'
from confluent_kafka import Producer
import json, time, random

producer = Producer({'bootstrap.servers': 'localhost:9092'})

# Injection de 200 articles 'fake urgents' simulant une nouvelle campagne
FAKE_TITLES = [
    'URGENT: Secret gouvernemental révélé - la vérité choquante !',
    'BREAKING: Découverte incroyable cachée au grand public',
    'EXCLUSIF: Ce que les médias ne vous diront jamais',
    'SCANDALE: Complot mondial enfin exposé',
    'CHOC: Preuves irréfutables du mensonge officiel',
]

for i in range(200):
    msg = {
        'id': f'test_drift_{i}',
        'title': random.choice(FAKE_TITLES) + f' (test {i})',
        'body': 'Contenu de test pour simulation de dérive',
        'source': 'test_drift',
        'source_category': 'suspicious',
        'language': 'fr',
        'timestamp': time.strftime('%Y-%m-%dT%H:%M:%SZ'),
        'gdelt_tone': -5.0, # Tonalité très négative
    }
    producer.produce('raw-news-stream', key=msg['id'],
value=json.dumps(msg))
```

```
time.sleep(0.05) # 20 messages/seconde

producer.flush()
print('200 messages de drift injectés !')
EOF

# Surveiller les alertes de drift
$ docker compose exec kafka kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic drift-alerts \
  --from-beginning
# Attendu (après ~30s) : alerte JSON avec composite_score > 0.5
```

✓ RÉSULTAT ATTENDU

Si vous voyez une alerte drift dans le topic 'drift-alerts' avec `composite_score > 0.5` : le pipeline fonctionne correctement !

Le score devrait monter progressivement : $0.0 \rightarrow 0.35 \rightarrow 0.50$ (ALERTE) $\rightarrow 0.80$ (DÉRIVE CONFIRMÉE).

Le learning rate devrait passer de $1e-05$ à $5e-05$ dans les logs Spark.

Le panneau P3 du dashboard Grafana devrait afficher la montée du score en temps réel.

**ÉTAPE
15****Démo de Soutenance : Scénario Pas à Pas**

Guide pour une démonstration exceptionnelle

**ASTUCE**

Ce scénario est conçu pour une démonstration de 10-15 minutes devant le jury.

Préparer 3 onglets ouverts : Kafdrop (9000), Grafana (3000), API Docs (8000/docs).

Le script de simulation peut être lancé à l'avance pour que des données soient déjà présentes.

Durée totale estimée du scénario guidé : 12 minutes.

Scénario de démonstration en 6 actes**ACTE 1 — Architecture et démarrage (2 min)**

- ▶ Montrer le fichier docker-compose.yml et expliquer les 11 services
- ▶ Lancer : `docker compose ps` pour montrer tous les services en cours
- ▶ Montrer le Spark Master UI (localhost:8080) : 2 workers actifs, job streaming running

ACTE 2 — Flux de données entrant (2 min)

- ▶ Ouvrir Kafdrop (localhost:9000) → topic raw-news-stream → montrer les messages en direct
- ▶ Expliquer : chaque message = un article de presse collecté par le scraper RSS
- ▶ Montrer la fréquence : ~20 messages/minute → pipeline en temps réel

ACTE 3 — Classification NLP (2 min)

- ▶ Appeler l'API : `curl http://localhost:8000/api/v1/articles/recent?limit=5`
- ▶ Montrer les champs `is_fake`, `confidence`, `p_fake` sur des exemples réels
- ▶ Ouvrir Grafana → Panneau P1 (Taux de Fakes) → expliquer la valeur temps réel
- ▶ Montrer l'API Swagger (localhost:8000/docs) → tester `/api/v1/search` avec un mot-clé
- ▶ Ouvrir Streamlit Dashboard (localhost:8501) → montrer la vue d'ensemble avec KPIs en direct

ACTE 4 — Déclenchement du Concept Drift (4 min — point CLEF)

```
# Lancer dans un terminal visible du jury :
$ python3 scripts/inject_drift_simulation.py
# Ce script injecte 200 articles fake simulant une nouvelle campagne virale
```

- ▶ Afficher Grafana en plein écran sur Panneau P3 (Score Composite Drift)
- ▶ Observer en direct la montée du score : 0.0 → 0.35 → 0.50 (zone orange) → 0.80 (zone rouge)

- ▶ Montrer le Panneau P5 (Learning Rate) : passage de 1e-5 à 5e-5 (adaptation automatique)
- ▶ Montrer les logs : `docker compose logs -f spark-app` → 'DRIFT ALERT : score=0.7X'
- ▶ Montrer l'API : `curl http://localhost:8000/api/v1/drift/events` → nouvel événement créé

ACTE 5 — Adaptation du modèle (2 min)

- ▶ Montrer dans les logs Spark la modulation du lr : '0.005 | lr: 5e-05' (vs 1e-05 avant)
- ▶ Expliquer : le Continual-DistilBERT se met à jour en continu sur les nouveaux articles
- ▶ Montrer le Panneau P7 (F1-Score Prequential) : maintien des performances malgré la dérive
- ▶ Comparer avec un modèle statique (graphique pre-généré) : dégradation progressive vs stabilité

ACTE 6 — Retour à la normale et synthèse (2 min)

- ▶ Montrer que le score drift redescend progressivement après la fin de l'injection
- ▶ Montrer le Panneau P2 (Flux d'articles) : retour à la normale
- ▶ Résumer : détection en < 8 minutes, adaptation automatique, F1 maintenu à 93 %

Script de simulation pour la soutenance

```
python
# scripts/inject_drift_simulation.py
# Script de simulation de Concept Drift pour la démonstration
from confluent_kafka import Producer
import json, time, random, argparse

parser = argparse.ArgumentParser()
parser.add_argument('--broker', default='localhost:9092')
parser.add_argument('--n', type=int, default=200, help='Nb articles à injecter')
parser.add_argument('--rate', type=float, default=0.1, help='Délai entre msgs (sec)')
args = parser.parse_args()

producer = Producer({'bootstrap.servers': args.broker})

# Titres simulant une nouvelle campagne de désinformation (thème inédit)
FAKE_TITLES_FR = [
    'URGENT — Découverte choquante sur les ondes 5G dissimulée par les autorités !',
    'EXCLUSIF : Le remède miracle censuré que Big Pharma veut effacer',
    'ALERTE : Preuve irréfutable du complot mondial enfin révélée',
    'SCANDALE : Ce que le gouvernement vous cache depuis des années',
    'BREAKING : Fuite de documents secrets explosifs sur la vaccination',
    'INCROYABLE : Scientifiques réduits au silence après leur
```

```
découverte',
    'CHOC : La vérité sur ce qui se passe vraiment en ce moment',
    'URGENT : Ceci va changer votre vie pour toujours – partagez !',
]

print(f'Injection de {args.n} articles fake vers {args.broker}...')
print('Surveiller Grafana (localhost:3000) → Panneau Score Drift')
print('Attendu : score > 0.5 après ~30-60 secondes')
print('-' * 60)

for i in range(args.n):
    msg = {
        'id': f'drift_sim_{int(time.time())}_{i}',
        'title': random.choice(FAKE_TITLES_FR),
        'body': 'Contenu simulant une nouvelle campagne de
désinformation.',
        'url': f'https://fake-news-test.example.com/article/{i}',
        'source': 'SimulationDrift',
        'source_category': 'suspicious',
        'language': 'fr',
        'timestamp': time.strftime('%Y-%m-%dT%H:%M:%SZ'),
        'gdelt_tone': round(random.uniform(-8.0, -3.0), 2),
    }
    producer.produce('raw-news-stream', key=msg['id'],
                     value=json.dumps(msg,
ensure_ascii=False).encode('utf-8'))
    if i % 10 == 0:
        producer.flush()
        print(f' [{i+1}/{args.n}] Messages envoyés...')
        time.sleep(args.rate)

producer.flush()
print(f'Injection terminée ! {args.n} articles envoyés.')
print('Vérifier : curl http://localhost:8000/api/v1/drift/events')
```

ÉTAPE 16 — Dashboard Streamlit

ÉTAPE 16 | Dashboard Streamlit — Interface Interactive Interface web interactive multi-page — Port 8501 — Durée estimée : 5 minutes

16.1 Présentation de Streamlit et rôle dans le projet

Streamlit est un framework open source Python permettant de créer des applications web interactives à partir de scripts Python purs, sans nécessiter de connaissances en HTML, CSS ou JavaScript. Il est particulièrement adapté aux projets de Data Science et de Machine Learning, car il permet d'exposer des visualisations de données complexes avec un minimum de code. Dans notre pipeline, Streamlit complète Grafana en offrant une interface plus riche et plus interactive : là où Grafana est optimisé pour les métriques temps réel, Streamlit permet l'exploration des données, la recherche dans les articles classifiés, et la visualisation des alertes de drift de façon plus lisible pour un utilisateur non-technicien.

Rôle de Streamlit dans le projet

- Dashboard principal multi-pages (5 vues)
- Vue d'ensemble : KPIs en temps réel (articles/heure, taux de fakes, score drift)
- Explorer les articles : recherche et filtrage des articles classifiés (MongoDB + Elasticsearch)
- Monitoring Drift : visualisation du score composite et des alertes ADWIN/KSWIN/PageHinkley
- Analyse des sources : répartition par source, fiabilité, évolution temporelle
- Configuration : paramètres live du pipeline (seuils, intervalles)

Port exposé : 8501 | Accès : <http://localhost:8501>

16.2 Arborescence du service Streamlit

```
streamlit-dashboard/ |— app.py          ← Application principale multi-pages |—
Dockerfile          ← Image python:3.12-slim |— requirements.txt ← streamlit, plotly,
pymongo, elasticsearch, pandas, requests
```

16.3 Fichier streamlit-dashboard/requirements.txt

```
text
# streamlit-dashboard/requirements.txt
streamlit==1.38.0
pymongo==4.9.1
elasticsearch==8.15.0
plotly==5.24.0
pandas==2.2.2
numpy==1.26.4
requests==2.32.3
python-dotenv==1.0.1
```

16.4 Dockerfile Streamlit (streamlit-dashboard/Dockerfile)

```
dockerfile
FROM python:3.12-slim
```

```
RUN apt-get update && apt-get install -y --no-install-recommends \
  curl && rm -rf /var/lib/apt/lists/*
```

```
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY app.py .
```

```
EXPOSE 8501
```

```
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
  CMD curl -sf http://localhost:8501/_stcore/health || exit 1
```

```
CMD ["streamlit", "run", "app.py", \
  "--server.port=8501", "--server.address=0.0.0.0", \
  "--server.headless=true", "--browser.gatherUsageStats=false"]
```

16.5 Ajout du service dans docker-compose.yml

```
yaml
#
# =====
# STREAMLIT DASHBOARD — Interface interactive
# Port 8501 — Toutes les visualisations en un seul endroit
#
# =====

streamlit:
  build:
    context: ./streamlit-dashboard
    dockerfile: Dockerfile
  container_name: streamlit
  restart: unless-stopped
  depends_on: [mongodb, elasticsearch, api]
  ports:
    - '8501:8501'
  environment:
    MONGO_URI: mongodb://mongodb:27017
    MONGO_DB: disinformation_db
    ES_HOST: http://elasticsearch:9200
    API_BASE: http://api:8000
    REFRESH_SEC: '30'
  deploy:
    resources:
      limits:
        memory: 512M
        cpus: '0.5'
```

16.6 Lancement et vérification

```
# Le service Streamlit est démarré automatiquement avec docker compose up -d
# Vérifier que le service démarre correctement :
$ docker compose logs streamlit | tail -20
# Attendu : "You can now view your Streamlit app in your browser."
```



```
# Accéder à l'interface :
$ xdg-open http://localhost:8501
# OU dans le navigateur : http://localhost:8501

# Vérifier l'état du container :
$ docker compose ps streamlit
# Attendu : streamlit Up (healthy)
```

16.7 Pages et fonctionnalités du Dashboard Streamlit

Page	Contenu / Fonctionnalité
 Vue d'ensemble	KPIs temps réel (articles/heure, taux fake %, score drift), graphiques Plotly, métriques système
 Explorer les articles	Recherche full-text dans Elasticsearch, filtres (source, langue, is_fake, période), tableau interactif
 Monitoring Drift	Score composite ADWIN+KSWIN+PageHinkley en temps réel, historique des alertes, learning rate courant
 Analyse des sources	Répartition par source (pie chart), fiabilité historique, évolution temporelle des articles par source
 Configuration	Paramètres live : seuils drift, intervalles scraping, refresh rate dashboard

Note importante : Streamlit fonctionne en complément de Grafana. Grafana est optimisé pour les métriques temps réel et les alertes opérationnelles. Streamlit offre une interface plus conviviale pour l'exploration des données et la démonstration lors de la soutenance.

Guide de Dépannage

Ce chapitre couvre les problèmes les plus fréquemment rencontrés lors de la mise en place du pipeline.

Problème 1 — Elasticsearch ne démarre pas (OOM error)

ERREUR FRÉQUENTE

Symptôme : container elasticsearch en état 'Exited (137)' ou 'OOM Killed'

```
# Vérifier les logs
$ docker compose logs elasticsearch | tail -20

# Solution 1 : Augmenter la mémoire virtuelle (Linux/WSL2)
$ sudo sysctl -w vm.max_map_count=262144

# Solution 2 : Réduire la JVM heap dans docker-compose.yml
# Changer ES_JAVA_OPTS=-Xms2g -Xmx2g vers ES_JAVA_OPTS=-Xms1g -Xmx1g
```

```
# Solution 3 : Sur Docker Desktop, augmenter la RAM allouée à 14 Go minimum
# Settings > Resources > Memory > 14 Go
```

Problème 2 — Spark ne se connecte pas à Kafka

✗ ERREUR FRÉQUENTE

Symptôme : 'Connection refused' ou 'LEADER_NOT_AVAILABLE' dans les logs Spark

```
$ docker compose logs spark-app | grep -i 'kafka\|error\|connection'

# Solution : Vérifier le nom du broker dans les variables d'environnement
# Dans docker-compose.yml, l'application Spark doit utiliser kafka:29092
# (port interne Docker) et non localhost:9092

# Vérifier la résolution DNS Docker
$ docker compose exec spark-app ping -c 3 kafka
# Attendu : réponses PING de l'IP interne de Kafka
```

Problème 3 — Erreur ImportError dans spark-app

✗ ERREUR FRÉQUENTE

Symptôme : ModuleNotFoundError: No module named 'transformers' dans les logs

```
# Solution : Rebuilder le container spark-app
$ docker compose build --no-cache spark-app
$ docker compose up -d spark-app

# Vérifier que le requirements.txt contient bien transformers
$ cat spark-app/requirements.txt | grep transformers
# Attendu : transformers==4.40.0
```

Problème 4 — Le modèle ONNX n'est pas trouvé

✗ ERREUR FRÉQUENTE

Symptôme : FileNotFoundError: /app/models/onnx/model_quantized.onnx

```
# Vérifier que le volume models est bien monté
$ docker compose exec spark-app ls /app/models/onnx/

# Si vide : le modèle n'a pas été entraîné ou le volume n'est pas correctement monté
# Vérifier dans docker-compose.yml :
# volumes:
#   - ./models:/app/models

# Relancer l'entraînement et l'export ONNX :
$ source venv_main/bin/activate
$ python scripts/train_model.py --epochs 10
```

```
$ python scripts/export_onnx.py
```

Problème 5 — Grafana ne reçoit pas de données

✗ ERREUR FRÉQUENTE

Symptôme : Panneaux Grafana affichent 'No data' ou 'Query failed'

```
# 1. Vérifier que Elasticsearch est accessible depuis Grafana
$ docker compose exec grafana curl http://elasticsearch:9200/_cat/indices
# Attendu : liste des index (articles, drift-events)

# 2. Vérifier que des données sont indexées
$ curl http://localhost:9200/articles/_count
# Si count=0 : le pipeline Spark n'indexe pas encore (attendre 2-3 min)

# 3. Reconfigurer la datasource Grafana
# Grafana > Configuration > Data Sources > Elasticsearch
# URL : http://elasticsearch:9200 (pas localhost !)
# Time field : processed_at
# Elasticsearch version : 8.x
```

Problème 6 — Drift non détecté lors de la simulation

✗ ERREUR FRÉQUENTE

Symptôme : Score composite reste à 0.0 malgré l'injection de messages fake

```
# Vérifier que les messages arrivent bien dans Kafka
$ docker compose exec kafka kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic raw-news-stream \
  --max-messages 5

# Vérifier que Spark consomme les messages
$ docker compose logs spark-app | tail -20 | grep 'Batch'

# Si le score ne monte pas : vérifier ADWIN_DELTA dans .env
# Une valeur trop élevée (ex: 0.1) rend ADWIN insensible
# Recommandé : ADWIN_DELTA=0.002

# Vérifier les seuils
$ grep DRIFT .env
# Attendu : DRIFT_COMPOSITE_THRESHOLD=0.5
```

Problème 7 — Erreurs d'installation liées à Python 3.12 (Résumé des corrections)

⚠ ATTENTION

Python 3.12 a introduit des changements internes majeurs (suppression de `longintrepr.h`,

refactorisation des APIs C). De nombreux anciens packages ne compilent plus directement. Ce tableau récapitule toutes les erreurs rencontrées et leurs solutions validées.

Erreur rencontrée	Cause	Solution validée
<code>python -m spacy download en_core_web_sm</code> → HTTP 404	Bug de formatage d'URL dans spaCy CLI avec Python 3.12	<code>pip install https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.7.1/en_core_web_sm-3.7.1-py3-none-any.whl</code>
<code>python -m spacy download fr_core_news_sm</code> → HTTP 404	Même cause que ci-dessus pour le modèle français	<code>pip install https://github.com/explosion/spacy-models/releases/download/fr_core_news_sm-3.7.0/fr_core_news_sm-3.7.0-py3-none-any.whl</code>
<code>sudo apt install git-lfs</code> → E: Le paquet <code>packettracer</code> doit être réinstallé	Paquet Cisco Packet Tracer corrompu bloquant apt	<code>sudo dpkg --remove --force-remove-reinstreq packettracer</code> → puis <code>sudo apt update</code> && <code>sudo apt install git-lfs -y</code>
<code>lxml==4.3.2</code> → fatal error: <code>longintrepr.h</code> : Aucun fichier	<code>longintrepr.h</code> supprimé dans Python 3.12 (CPython)	<code>pip install lxml>=5.2.2</code> (recompilé pour Python 3.12)
<code>Pillow==5.4.1</code> → <code>RequiredDependencyException: jpeg</code>	Ancienne version incompatible Python 3.12 + <code>libjpeg</code>	<code>sudo apt install libjpeg-dev libpng-dev</code> → <code>pip install Pillow>=10.3.0</code>
<code>feedparser==5.2.1</code> → error: <code>use_2to3</code> is invalid	<code>feedparser 5.x</code> utilise <code>setup.py use_2to3</code> déprécié en Python 3.12	Éditer <code>requirements.txt</code> : remplacer <code>feedparser==5.2.1</code> par <code>feedparser>=6.0.11</code>
<code>newspaper3k==0.2.8</code> → multiples erreurs de compilation	<code>newspaper3k</code> abandonné en 2020, incompatible Python 3.12	<code>pip install newspaper4k>=0.9.3.1</code> (fork maintenu, API identique)
<code>asyncio==3.4.3</code> → package redondant	<code>asyncio</code> est intégré dans Python 3.4+ (standard library)	Retirer <code>asyncio</code> du <code>requirements.txt</code> (pas besoin de <code>pip install</code>)

Problème 8 — asyncio package inutile en Python 3.12

INFO

`asyncio` fait partie de la bibliothèque standard Python depuis la version 3.4.

Il ne doit PAS être installé via `pip`. La commande `pip install asyncio==3.4.3` échouera ou installera une version obsolète potentiellement conflictuelle.

```
# ❌ INCORRECT – Ne pas faire :
# $ pip install asyncio==3.4.3

# ✅ CORRECT – asyncio est déjà disponible nativement :
$ python -c "import asyncio; print(asyncio.__version__)"
```

```
# Sur Python 3.12 : pas de numéro de version distinct (stdlib intégrée)

# Pour utiliser asyncio dans vos scripts, simplement importer :
# import asyncio ← c'est tout, aucune installation requise
```

Problème 9 — Requirements FakeNewsNet incompatibles avec Python 3.12

Le dépôt GitHub officiel KaiDMML/FakeNewsNet contient un requirements.txt datant de 2019, incompatible avec Python 3.12. Voici le fichier requirements.txt corrigé à utiliser en remplacement :

```
text
# requirements.txt CORRIGÉ pour Python 3.12
# À utiliser à la place du requirements.txt original du dépôt FakeNewsNet

beautifulsoup4>=4.12.0
certifi>=2024.2.2
chardet>=5.2.0
click>=8.1.7
cssselect>=1.2.0
feedfinder2>=0.0.4
feedparser>=6.0.11
Flask>=3.0.3
Flask-Cors>=4.0.1
idna>=3.7
lxml>=5.2.2
newspaper4k>=0.9.3.1
nltk>=3.8.1
Pillow>=10.3.0
python-dateutil>=2.9.0
PyYAML>=6.0.1
requests>=2.32.3
requests-file>=1.5.1
six>=1.16.0
soupsieve>=2.5
tldextract>=5.1.2
tqdm>=4.66.4
urllib3>=2.2.1
```

```
# Pour appliquer ce fichier corrigé :
$ cd ~/desinformation-pipeline/data/raw/fakenewsnet

# 1. Installer les dépendances système requises
$ sudo apt-get install -y libxml2-dev libxslt1-dev libjpeg-dev libpng-dev

# 2. Remplacer le requirements.txt par la version corrigée (voir ci-dessus)
# 3. Installer
$ pip install -r requirements.txt
```

```
# OU utiliser directement la Méthode HuggingFace (recommandée, sans
dépendances C) :
$ python -c "from datasets import load_dataset;
ds=load_dataset('GonzaloA/fake_news'); ds.save_to_disk('.')"
```

Problème 10 — Fakeddit inaccessible sur toutes les sources en 2025

✗ ERREUR FRÉQUENTE

Symptôme : git clone donne fichiers TSV de 0 octet, Zenodo retourne 404,
HuggingFace retourne DatasetNotFoundError, GitHub FakeNewsNet/fakeddit introuvable.

```
# NE PAS retenter ces commandes (toutes échouent en 2025) :
# git clone https://github.com/entitize/Fakeddit.git → fichiers 0 octet
# wget 'https://zenodo.org/record/4068222/files/all_train.tsv' → 404
# load_dataset('fakeddit') → DatasetNotFoundError
# git clone https://github.com/FakeNewsNet/fakeddit.git → Repository not
found
```

```
# ✓ SOLUTION : Fakeddit est définitivement retiré du projet.
# Les 4 datasets restants (~153 000 exemples) sont amplement suffisants.
# Le script preprocess_data.py corrigé n'inclut plus Fakeddit.
# Passer directement à : python scripts/preprocess_data.py
```

Problème 11 — FakeNewsNet : code/process_data/ introuvable

✗ ERREUR FRÉQUENTE

Symptôme : python3.12 code/process_data/get_politifact_data.py → No such file or directory

```
# Cause : Le dépôt KaiDMML/FakeNewsNet contient les données déjà prêtes
dans dataset/
# Les scripts de collecte requièrent des clés API Twitter désactivées
depuis 2023.
```

```
# ✓ SOLUTION : Ne pas exécuter les scripts, utiliser les CSV
directement.
$ ls data/raw/fakenewsnet/dataset/
# Attendu : gossipcop_fake.csv gossipcop_real.csv politifact_fake.csv
politifact_real.csv
# Le script preprocess_data.py (version corrigée) charge ces CSV
automatiquement.
```

Problème 12 — train_model.py : FileNotFoundError data/processed/train/train.csv

✗ ERREUR FRÉQUENTE

Symptôme : FileNotFoundError: [Errno 2] No such file or directory:
'data/processed/train/train.csv'

```
# Cause : preprocess_data.py n'a pas encore été exécuté.
```

```
#  SOLUTION — Respecter l'ordre impératif (depuis la RACINE, venv_main
activé) :
$ cd ~/desinformation-pipeline && source venv_main/bin/activate

# Étape 1 : Télécharger les datasets
$ bash scripts/download_datasets.sh

# Étape 2 : Générer les CSV prétraités (OBLIGATOIRE avant train_model.py)
$ python scripts/preprocess_data.py
$ ls data/processed/train/ # Doit afficher train.csv

# Étape 3 : Seulement APRÈS l'étape 2
$ python scripts/train_model.py --epochs 10 --batch_size 64 --lr 2e-5
```

Problème 13 — ModuleNotFoundError: No module named 'datasets' hors venv

ERREUR FRÉQUENTE

Symptôme : ModuleNotFoundError: No module named 'datasets'

Cause : venv_main non activé avant l'appel Python.

```
#  SOLUTION : Activer venv_main avant tout appel Python :
$ source venv_main/bin/activate
$ echo $VIRTUAL_ENV
# Attendu : /home/<user>/desinformation-pipeline/venv_main
```

Si datasets n'est pas installé dans venv_main :

```
$ pip install datasets==2.21.0
```

Problème 14 — chmod: impossible d'accéder à 'scripts/download_datasets.sh'

ERREUR FRÉQUENTE

Symptôme : chmod: impossible d'accéder à 'scripts/download_datasets.sh': Aucun fichier

Cause : La commande est lancée depuis un sous-dossier (ex: data/raw/liar/)

au lieu de la racine du projet.

```
#  SOLUTION : Toujours revenir à la racine AVANT toute commande :
$ cd ~/desinformation-pipeline
$ pwd
# Attendu : /home/<user>/desinformation-pipeline
$ chmod +x scripts/download_datasets.sh
$ bash scripts/download_datasets.sh
```

Problème 15 — RuntimeError: expected scalar type Float but found Double

✖ ERREUR FRÉQUENTE

Symptôme : RuntimeError: expected scalar type Float but found Double

Traceback dans : `loss = loss_fn(out.logits, lbls)` dans `train_model.py`

```
# CAUSE : Deux bugs de types dans train_model.py :

# Bug 1 – Le vecteur de poids w est en Float64 par défaut :
# w = torch.tensor([...]).to(DEVICE) ← INCORRECT (Float64)

# Bug 2 – Les labels lbls peuvent ne pas être en int64 :
# lbls = batch['labels'].to(DEVICE) ← RISQUE (type dépend du Dataset)

# ✅ CORRECTIONS :
# Bug 1 : ajouter .float() pour forcer Float32
# w = torch.tensor([...]).float().to(DEVICE) ← CORRECT

# Bug 2 : ajouter .long() pour forcer int64
# lbls = batch['labels'].to(DEVICE).long() ← CORRECT

# RÈGLE CrossEntropyLoss :
# - input (logits) → Float32 (float)
# - target (labels) → int64 (long)

# ✅ Script train_model.py corrigé : utiliser la version v2 fournie en
Étape 5.1
# (le script actuel dans la documentation intègre déjà ces deux
corrections)

# Si tu as l'ancienne version, appliquer ces deux corrections
manuellement :
# Ligne des poids :
# AVANT : w = torch.tensor([n_fake/n_total, n_real/n_total]).to(DEVICE)
# APRÈS : w = torch.tensor([n_fake/n_total,
n_real/n_total]).float().to(DEVICE)

# Ligne des labels dans la boucle :
# AVANT : lbls = batch['labels'].to(DEVICE)
# APRÈS : lbls = batch['labels'].to(DEVICE).long()
```

Problème 16 — Entraînement très lent sur CPU (batch_size=64)

i INFO

Symptôme : Epoch 1/10: 0% | 0/1211 [00:11<?, ?it/s] → très lent

```
# Sur CPU, batch_size=64 charge trop de données simultanément.
# Chaque batch prend ~6-11 secondes → 1211 batches * 8s ≈ 2h45 par epoch

# ✅ SOLUTION 1 : Réduire le batch_size
```



```
$ python scripts/train_model.py --epochs 5 --batch_size 16 --lr 2e-5
# batch_size=16 : ~1.5s/batch, bien plus raisonnable sur CPU

# ✅ SOLUTION 2 (recommandée) : Utiliser Google Colab avec GPU T4 gratuit
# https://colab.research.google.com → Runtime > T4 GPU
# Durée sur T4 : ~45 min pour 77 000 exemples / 10 epochs

# ✅ SOLUTION 3 : Réduire les epochs pour un test rapide
$ python scripts/train_model.py --epochs 3 --batch_size 16 --lr 2e-5
```

Problème 17 — curl: (23) Failed writing body — Dockerfile spark-app

✗ ERREUR FRÉQUENTE

Symptôme : curl: (23) Failed writing body (0 != 1360)

Warning: Failed to create the file /opt/bitnami/spark/jars/spark-sql-kafka.jar: No such file or directory

exit code: 23

```
# CAUSE : Utilisation de l'image apache/spark:3.5.1 au lieu de
bitnami/spark:3.5
#
# apache/spark:3.5.1 → dossier jars = /opt/spark/jars/ (DIFFÉRENT)
#                    → Python 3.8 (INCOMPATIBLE avec nos packages 3.12)
# bitnami/spark:3.5  → dossier jars = /opt/bitnami/spark/jars/
# (CORRECT)
#                    → Python 3.12 (compatible avec nos packages)

# ✅ SOLUTION : Vérifier et corriger le Dockerfile de spark-app
# Ouvrir spark-app/Dockerfile et vérifier la première ligne :
$ head -1 spark-app/Dockerfile
# DOIT afficher : FROM bitnami/spark:3.5
# Si affiche : FROM apache/spark:3.5.1 → CORRIGER cette ligne

# Correction rapide :
$ sed -i 's|FROM apache/spark:3.5.1|FROM bitnami/spark:3.5|g'
spark-app/Dockerfile
$ sed -i 's|FROM apache/spark:.*|FROM bitnami/spark:3.5|g'
spark-app/Dockerfile

# Aussi remplacer curl -L par curl -fsSL pour détecter les erreurs
proprement :
$ grep 'curl' spark-app/Dockerfile
# DOIT contenir : mkdir -p /opt/bitnami/spark/jars && curl -fsSL ...

# Rebuilder après correction :
$ docker compose build --no-cache spark-app
$ docker compose up -d spark-app
```

Problème 18 — asyncio==3.4.3 dans producer/requirements.txt

✗ ERREUR FRÉQUENTE

Symptôme : asyncio==3.4.3 installé dans le container producer (Warning ou conflit)

```
# CAUSE : pip freeze a capturé asyncio dans requirements.txt mais
# asyncio est une bibliothèque standard Python (stdlib), pas un package
pip.
```

```
# ✅ SOLUTION : Supprimer asyncio du fichier requirements.txt
$ sed -i '/^asyncio/d' producer/requirements.txt
```

```
# Vérification :
$ grep asyncio producer/requirements.txt
# Attendu : aucune ligne (asyncio supprimé)
```

```
# Puis rebuild le container producer :
$ docker compose build --no-cache rss-producer
```

Annexe — Récapitulatif des Commandes Essentielles

Commandes de gestion Docker

Action	Commande
Démarrer tous les services	<code>docker compose up -d</code>
Arrêter tous les services	<code>docker compose down</code>
Voir le statut de tous les services	<code>docker compose ps</code>
Voir les logs d'un service	<code>docker compose logs -f [service]</code>
Redémarrer un service	<code>docker compose restart [service]</code>
Rebuild une image	<code>docker compose build --no-cache [service]</code>
Supprimer tous les volumes (reset total)	<code>docker compose down -v</code>
Ouvrir un shell dans un container	<code>docker compose exec [service] bash</code>
Voir les ressources utilisées	<code>docker stats</code>

Commandes de vérification des services

Service	Commande de vérification
Kafka topics	<code>docker compose exec kafka kafka-topics --list --bootstrap-server localhost:9092</code>
Kafka consumer	<code>docker compose exec kafka kafka-console-consumer --bootstrap-server localhost:9092 --topic raw-news-stream --max-messages 3</code>
MongoDB stats	<code>docker compose exec mongodb mongosh disinformation_db --eval 'db.articles.countDocuments()'</code>
Elasticsearch health	<code>curl http://localhost:9200/_cluster/health?pretty</code>
Elasticsearch count	<code>curl http://localhost:9200/articles/_count</code>
API stats	<code>curl -s http://localhost:8000/api/v1/stats python3 -m json.tool</code>
API articles	<code>curl 'http://localhost:8000/api/v1/articles/recent?limit=5'</code>
Drift events	<code>curl 'http://localhost:8000/api/v1/drift/events'</code>

URLs des interfaces web

Interface	URL	Identifiants
Kafdrop (Kafka UI)	<code>http://localhost:9000</code>	—
Spark Master	<code>http://localhost:8080</code>	—
Grafana Dashboard	<code>http://localhost:3000</code>	admin / admin2025
API FastAPI (Swagger)	<code>http://localhost:8000/docs</code>	—

API FastAPI (ReDoc)	http://localhost:8000/redoc	—
---------------------	-----------------------------	---

Commandes Python utiles

Action	Commande
Activer venv principal	source venv_main/bin/activate
Télécharger les datasets	bash scripts/download_datasets.sh
Prétraiter les données	python scripts/preprocess_data.py
Entraîner le modèle	python scripts/train_model.py --epochs 10 --batch_size 16
Exporter en ONNX INT8	python scripts/export_onnx.py
Lancer la simulation de drift	python scripts/inject_drift_simulation.py --n 200 --rate 0.1
Lancer les notebooks	jupyter lab notebooks/

Table de compatibilité Python 3.12 — Versions validées 2025

Ce tableau récapitule toutes les versions de packages validées sur Python 3.12 (Ubuntu 24.04). À utiliser comme référence lors de tout pip install pour ce projet.

Package	Version validée Python 3.12	Remarque
torch	2.4.0	wheels Python 3.12 disponibles depuis PyTorch 2.2+
transformers	4.44.0	HuggingFace, stable Python 3.12
datasets	2.21.0	HuggingFace, compatible
accelerate	0.34.0	Stable Python 3.12
pyspark	3.5.3	PySpark 3.5.x supporte Python 3.12
river	0.21.2	ADWIN/KSWIN/PageHinkley, Python 3.12 OK
onnxruntime	1.19.0	Wheels Python 3.12 disponibles
optimum	1.21.0	Export ONNX, Python 3.12 OK
spacy	3.7.6	Installer avec pip install spacy==3.7.6
en_core_web_sm	3.7.1	Via wheel direct GitHub (pas python -m spacy download)
fr_core_news_sm	3.7.0	Via wheel direct GitHub (pas python -m spacy download)
newspaper3k	✗ ABANDONNÉ	Utiliser newspaper4k >= 0.9.3.1 à la place
newspaper4k	0.9.3.1	Remplaçant officiel de newspaper3k, Python 3.12 OK
lxml	5.2.2+	lxml 4.x incompatible P3.12 (longintrepr.h

		supprimé)
Pillow	10.3.0+	Pillow 5.x/6.x incompatible P3.12, utiliser ≥ 10
feedparser	6.0.11	feedparser 5.x incompatible P3.12 (use_2to3)
asyncio	STDLIB — NE PAS INSTALLER	Intégré nativement en P3.12+. Supprimer du requirements.txt.
confluent-kafka	2.5.0	librdkafka bundlée, Python 3.12 OK
pymongo	4.9.1	Stable Python 3.12
elasticsearch	8.15.0	Client ES 8.x, Python 3.12 OK
pandas	2.2.2	LTS, stable Python 3.12
numpy	1.26.4	Dernière version 1.x, stable P3.12
scikit-learn	1.5.2	Stable Python 3.12
mlflow	2.15.1	Stable Python 3.12
jupyterlab	4.2.5	Utiliser jupyterlab (pas jupyter classic)
fastapi	0.113.0	ASGI, stable Python 3.12
uvicorn	0.30.6	ASGI server, Python 3.12 OK
gdown	5.2.0	Téléchargement Google Drive, Python 3.12 OK
pydantic	2.9.2	Validation Pydantic v2, Python 3.12 OK

Table de compatibilité des images Docker

✗ ERREUR FRÉQUENTE

Ne jamais substituer apache/spark:X.X.X à bitnami/spark:3.5 !

apache/spark:3.5.1 → Python 3.8 (incompatible) + /opt/spark/jars/ (chemin différent)

bitnami/spark:3.5 → Python 3.12 (compatible) + /opt/bitnami/spark/jars/ (correct)

Image Docker	Python inclus	Dossier JARs Spark	Usage dans ce projet
bitnami/spark:3.5	Python 3.12	/opt/bitnami/spark/jars/	spark-master, spark-worker-1, spark-worker-2, spark-app
apache/spark:3.5.1	Python 3.8	/opt/spark/jars/	NE PAS UTILISER — Python 3.8 trop ancien
python:3.12-slim	Python 3.12	—	producer (rss-producer), api (FastAPI)
python:3.12-slim	Python 3.12	—	NE PAS UTILISER — remplacé par 3.12-slim
confluentinc/cp-kafka:7.6.0	—	—	Kafka broker
confluentinc/cp-zookeeper:7.6.0	—	—	ZooKeeper

mongo:7.0	—	—	MongoDB
docker.elastic.co/ elasticsearch/ elasticsearch:8.14.0	—	—	Elasticsearch
grafana/grafana:10.4.0	—	—	Grafana Dashboard
obsidiandynamics/ kafdrop:latest	—	—	Kafka UI

Corrections de packages FakeNewsNet (requirements.txt Python 3.12)

Package original (❌ incompatible)	Remplaçant Python 3.12 (✅)	Raison
lxml==4.3.2	lxml>=5.2.2	longintrepr.h supprimé en Python 3.12
Pillow==5.4.1	Pillow>=10.3.0	API C interne refactorisée en Python 3.12
feedparser==5.2.1	feedparser>=6.0.11	use_2to3 invalide en Python 3.12
newspaper3k==0.2.8	newspaper4k>=0.9.3.1	newspaper3k abandonné, incompatible
Flask==1.0.2	Flask>=3.0.3	Werkzeug incompatible avec Flask 1.x sur Python 3.12
asyncio==3.4.3	SUPPRIMER du requirements.txt	stdlib Python, jamais pip install

✅ RÉSULTAT ATTENDU

Pipeline opérationnel ! Résumé des accès :

- Données temps réel : <http://localhost:3000> (Grafana — admin/admin2025)
- Monitoring Kafka : <http://localhost:9000> (Kafdrop)
- API REST : <http://localhost:8000/docs> (Swagger UI)
- Spark Master UI : <http://localhost:8080>

Pour la soutenance, lancer depuis la racine (venv_main activé) :

```
python scripts/inject_drift_simulation.py --n 200 --rate 0.1
```

Images Docker: bitnami/spark:3.5 + python:3.12-slim (PAS apache/spark ni python:3.12)

Toutes versions compatibles Python 3.12 (Ubuntu 24.04 Noble).

— Fin de la Documentation Technique —

KOMOSSI Sosso — Master BIG DATA IA — UCAO UUT — 2025-2026

Mises à jour Version 2.0
Mises à jour Version 2.0

Mises à jour Version 2.0

Les modifications suivantes ont été apportées au projet dans le cadre de la version 2.0 (avril 2026). Elles améliorent la robustesse du modèle, l'accessibilité de l'interface et l'optimisation des ressources.

● Équilibrage du dataset d'entraînement

Le corpus est désormais sous-échantillonné pour garantir 50 % d'articles faux et 50 % d'articles vrais avant entraînement. Le split est passé de 80/10/10 à 60/20/20 (train/validation/test). Un `WeightedRandomSampler` assure des mini-batches équilibrés même en cas de légère imbalance résiduelle.

● Reservoir d'apprentissage continu équilibré par classe

Le buffer de replay (Reservoir Sampling) est maintenant divisé en deux sous-buffers indépendants : 2 500 exemples faux + 2 500 exemples vrais. Même lors d'injections massives de désinformation (concept drift, scénario B), le modèle conserve un ratio fake/réel de 50/50 lors de la mise à jour en ligne.

● Poids de classe corrigés dans la loss en ligne

La fonction de perte `CrossEntropyLoss` utilise désormais des poids inversement proportionnels à la fréquence de chaque classe dans le mini-batch. La classe minoritaire reçoit plus de signal, empêchant le modèle de se spécialiser uniquement dans la détection des faux articles.

● Port Spark UI exposé (port 4040)

L'interface web de monitoring Apache Spark est maintenant accessible sur le port 4040. Elle permet de visualiser les jobs Spark Streaming en cours, les DAG d'exécution, les statistiques de traitement et les métriques des micro-batches en temps réel.

● Optimisation mémoire — machine 12 GB RAM

La consommation mémoire totale a été réduite de ~2.3 GB pour permettre l'ouverture d'un navigateur (Firefox) pendant l'exécution du pipeline :

- Spark driver JVM : 1 500 m → 1 200 m (-300 MB)
- Spark container limit : 4 GB → 3 GB (-1 GB)
- Elasticsearch JVM : 384 m → 256 m (-128 MB)
- Elasticsearch container : 768 M → 512 M (-256 MB)
- Drift injector interval : 15 min → 30 min (-50 % pression CPU/RAM)

● Correctifs Grafana — Alertes

Le message 'Unknown state execution_error' est résolu : le paramètre `execErrState` est passé de 'Error' à 'Alerting' dans toutes les règles d'alerte. Le panneau d'alertes affiche maintenant correctement les 5 règles configurées (Concept Drift détecté, Drift confirmé, Taux fake >70 %, Taux fake >40 %, Confiance modèle faible <70 %).

● Contexte africain renforcé

La documentation et l'interface Streamlit (page À propos) soulignent explicitement le déploiement en contexte africain (Afrique subsaharienne). Les sources prioritaires sont AFP Afrique, RFI, Jeune Afrique, Al Jazeera, France24, VOA News Africa. L'architecture est universelle et exportable hors d'Afrique.

Mises à jour Version 2.1

La version 2.1 (avril 2026) apporte la fonctionnalité de recherche web en temps réel avec classification instantanée, corrige la stabilité long-terme de Spark, améliore le monitoring Grafana et renforce la résilience du pipeline.

● Recherche Web temps réel — Nouveau endpoint API

Ajout de GET `/api/v1/search/web` : recherche d'articles d'actualité sur internet avec classification immédiate par le modèle DistilBERT ONNX INT8.

- Source primaire : Google News RSS (gratuit, sans clé API, aucun rate-limit agressif)
- Source secondaire (fallback) : DuckDuckGo News
- Inférence ONNX directe dans l'API — résultats en < 10 ms/article
- Les articles sont aussi publiés vers Kafka pour l'apprentissage continu de Spark
- Persistance MongoDB des résultats classifiés
- Cache in-memory 5 minutes : la même requête ne ré-interroge pas le web
- Mécanisme anti-ratelimit DuckDuckGo : délai minimum 4 s entre appels, retry 3× avec backoff 5 s / 20 s / 45 s
- Endpoint complémentaire : GET `/api/v1/search/web/sources`

● Interface Streamlit — Page Recherche refonctionnée (2 onglets)

La page Recherche du dashboard Streamlit est maintenant organisée en deux onglets :

- Onglet 1 —  Recherche Web en direct : saisie libre → Google News RSS → classification ONNX instantanée → badges FAKE/RÉEL + score de confiance + liens cliquables vers les articles originaux + graphique des probabilités + export CSV. Sujets suggérés contextualisés pour l'Afrique.
- Onglet 2 —  Base de données : recherche full-text Elasticsearch sur les articles déjà indexés, avec scatter plot de pertinence.

Les articles web analysés enrichissent automatiquement l'apprentissage du modèle.

● Architecture API enrichie — ONNX dans le container API

Le container API embarque désormais le runtime d'inférence ML :

- onnxruntime==1.19.2 installé dans l'image Docker API
- transformers==4.44.2 + tokenizers==0.19.1 pour le tokenizer DistilBERT
- Volume `./models/onnx` monté en lecture seule dans le container API
- Chargement lazy du modèle (une seule fois au premier appel `/search/web`)
- 1 worker uvicorn au lieu de 2 (évite double chargement ONNX 130MB → OOM)
- Mémoire container API : 256 MB → 768 MB pour accueillir le runtime ONNX

● Correctif Grafana — Plage temporelle et performances

Résolution du problème 'No Data' sur tous les panels Grafana :

- Plage par défaut : now-1h → now-24h (les données sont bien présentes)
- Refresh automatique : 5 s → 30 s (réduit la charge CPU sur machine 12 GB)
- Timepicker enrichi : options 1h, 3h, 6h, 12h, 24h, 2j, 7j, 30j
- Cause identifiée : avec Spark traitant des batches toutes les ~15 min, la fenêtre 1h était souvent vide entre deux batches

● Stabilisation Spark — Correctif crash Py4JError (OOM JVM)

Résolution du crash Spark 'Py4JError: An error occurred while calling awaitTermination' survenant après 12+ heures de fonctionnement :

- Cause : la JVM (Java Virtual Machine) atteignait l'OOM (Out of Memory) par accumulation de garbage non collecté avec le GC par défaut (Parallel GC)
- Correction : activation du G1GC (Garbage First Garbage Collector) :
 - XX:+UseG1GC -XX:G1HeapRegionSize=4m -XX:MaxGCPauseMillis=500
 - XX:InitiatingHeapOccupancyPercent=35 -XX:+ExplicitGCInvokesConcurrent

- maxOffsetsPerTrigger : 1 000 → 200 (batches plus légers, moins de pression mémoire)
- Impact : Spark fonctionne maintenant en continu sans crash

● RSS Producer — Résilience et continuité du flux de données

Amélioration de la continuité du flux RSS vers Kafka :

- Identification du problème : après 5-6 jours sans redémarrage, tous les articles RSS sont dédupliqués (seen_ids set) et le producer envoie 0 article/cycle
- Solution : redémarrage périodique du container pour réinitialiser le cache
- Au redémarrage : envoi immédiat de ~150 articles (12 sources × 20 articles max)
- Spark reprend le traitement dans les 5 minutes suivant le redémarrage du producer

● Performances v2.1 — Métriques mises à jour

Résultats du modèle Continual-DistilBERT v2.1 (après correction du biais) :

- F1-score macro (validation) : 98.49 % (amélioration vs 93.2 % initial)
- F1-score classe réel (0) : ≥ 88 % (objectif early stopping)
- F1-score classe fake (1) : ≥ 88 % (objectif early stopping)
- AUC-ROC : 99.89 %
- Latence inférence ONNX INT8 : ~5-6 ms/article (Spark) / < 10 ms (API web search)
- Taux de compression FP32 → INT8 : ~75 %
- Débit pipeline : ~200 articles/batch/5 s
- Articles traités au 30 avril 2026 : > 10 000
- Taux de fake détecté en production : ~87-94 % (drift injector actif)

Mises à jour Version 2.2

La version 2.2 (avril 2026) résout le blocage du monitoring Grafana causé par la saturation mémoire de Spark et le swap excessif. Les batches passent de 17-20 minutes à 30-35 secondes grâce à l'entraînement conditionnel, SGD optimisé et la correction de la déduplication RSS.

● Résolution du blocage Grafana — Pipeline ralenti par saturation mémoire

Diagnostic : le container Spark consommait 3 GiB / 3 GiB (100 % de sa limite mémoire). Le système utilisait 3.4 GiB de swap sur disque USB externe (30 MB/s), rendant chaque batch Spark de 197 articles 17 à 20 minutes. Grafana recevait de nouvelles données seulement toutes les 20 minutes.

Corrections apportées :

- Mémoire Spark : 3 G → 3.5 G dans docker-compose.yml (headroom suffisant)
- Mode eval/train PyTorch séparé : `pt_model.eval()` entre les entraînements
- Softmax ONNX en pur numpy (zéro tenseur PyTorch dans la boucle d'inférence)

Résultat : 2.28 GiB / 3.42 GiB (67 %), plus aucun swap sur le chemin critique.

● Entraînement continu optimisé — 1 entraînement tous les 5 batches

Le PyTorch DistilBERT (66M paramètres) + SGD (momentum) totalise ~528 MB de données PyTorch. L'entraînement à chaque batch causait 248s de chargement depuis le swap. Solution : variable d'environnement `ONLINE_TRAIN_EVERY_N` (défaut 5).

- Batches 1-4 : inference seule = 30-35s/batch
- Batch 5 (entraînement) : ~248s
- Temps moyen : ~76s/batch (vs 17-20 min avant)
- L'apprentissage continu reste actif mais n'impacte plus le flux temps réel.

● Remplacement AdamW par SGD (momentum) — économie 264 MB

AdamW stockait deux moments (m et v) soit $2 \times 264 \text{ MB} = 528 \text{ MB}$ d'état optimizer.

Remplacement par SGD avec momentum (1 seul vecteur momentum = 264 MB).

Économie : 264 MB de RAM permanente dans le container Spark. La qualité de l'apprentissage en ligne est maintenue grâce au momentum SGD (0.9).

● RSS Producer — Cache de déduplication avec TTL 2 heures

Problème : le set `seen_ids` (set Python infini) retenait tous les IDs vus depuis le démarrage. Après le premier cycle (151 articles), 0 article était envoyé pendant des heures. Solution : remplacement du set par un dict `{id: timestamp}` avec TTL de 7200s (variable `RSS_DEDUP_TTL_SEC`). Les articles plus vieux que 2h expirent et peuvent être renvoyés vers Kafka pour alimenter le pipeline d'apprentissage continu. MongoDB et Elasticsearch utilisent l'upsert — aucun doublon en base.

● Monitoring temps réel restauré — Grafana alimenté toutes les 30-60 s

Avec les optimisations v2.2, le pipeline fonctionne désormais en quasi-temps réel :

- Spark traite ~100 articles toutes les 30-35s (batches sans entraînement)
- Grafana (refresh 30s) affiche les nouvelles données à chaque cycle
- ~300 articles indexés par heure en production normale
- Résultat diagnostiqué par profiling : `collect=1s`, `inference=28s`, `train=0s`
- Les logs de timing (`collect=`, `boucle inference=`, `train=`) sont conservés pour permettre la surveillance continue des performances.

Mises à jour Version 2.3

La version 2.3 (juillet 2026) corrige les faux positifs de classification sur les sources d'information officielles, améliore la compatibilité avec le déploiement cloud Streamlit, et garantit la cohérence des chemins de volumes Docker après migration du projet.

● Correction faux positifs — Seuil de décision 0.5 → 0.65

Problème identifié : des articles de sources officielles (AFP, Reuters, BBC, RFI) étaient classifiés comme faux (`is_fake=1`) en raison d'un décalage de domaine entre les données d'entraînement américaines (ISOT, WELFake) et les flux RSS africains/francophones.

Solution appliquée : le seuil de décision `p_fake` est relevé de 0.50 à 0.65 dans `nlp_classifier.py` (spark-app) et `web_search.py` (api). Un article est désormais classifié comme faux uniquement si $P(\text{fake}) \geq 0.65$, réduisant significativement les faux positifs sur les dépêches officielles tout en maintenant une haute sensibilité aux vraies désinformations.

● Déploiement Streamlit Cloud — Compatibilité Python 3.14

Streamlit Cloud utilise Python 3.14.6 par défaut et ignore `runtime.txt`. Les packages épinglés à des versions anciennes sans wheel Python 3.14 (`pillow==10.4.0`, `pandas==2.2.2`) provoquaient un échec de compilation. Solution : contraintes souples (`>=`) sur toutes les dépendances ; suppression de `pillow` du `requirements.txt` (inclus automatiquement par streamlit).

L'application intègre un mode démonstration (variable `IS_CLOUD`) qui désactive les appels TCP vers les services locaux non disponibles dans l'environnement cloud.

● Reconstruction des containers Docker — Migration Disque local5 → local6

Après migration du projet du disque `local5` vers `local6`, les containers Docker conservaient les anciens chemins de volumes liés (`bind mounts`). Les services `spark-app`, `api`, `rss-producer` et `drift-injector` pointaient vers `/media/florent/Disque local5/` et échouaient au démarrage.

Correction : `docker compose stop + rm` puis `docker compose up -d` pour les quatre services concernés. Les volumes nommés Docker (`mongodb_data`, `elasticsearch_data`, `kafka_data`, etc.) n'ont pas été affectés — aucune perte de données.

Mises à jour Version 2.4

La version 2.4 (juillet 2026) corrige un bug critique responsable d'un taux de fake anormalement élevé sur la recherche web, décontamine les statistiques polluées par des données de simulation, renforce le contexte africain multilingue du corpus d'entraînement, et introduit un cycle de dérive non permanent (fenêtre de visualisation puis retour automatique à la normale).

● Bug critique corrigé — désynchronisation du format d'entrée en recherche web

Diagnostic : `api/src/routers/web_search.py` construisait le texte à classifier par simple concaténation ("`{title} {body}`"), alors que l'entraînement (`scripts/train_model.py`) et le classifieur Spark (`spark-app/src/nlp_classifier.py`) utilisent le format "`{title[:200]} [SEP] {body[:100]}`". Cette divergence plaçait chaque recherche web hors de la distribution d'entraînement du modèle. Vérification empirique sur un même article réel de l'AFP : `p_fake` = 6,6 % avec le format d'entraînement contre 60,1 % avec l'ancien format de `web_search.py`. Correction : `web_search.py` reproduit désormais exactement le format d'entraînement.

● Décontamination MongoDB / Elasticsearch — statistiques faussées depuis des semaines

L'ancien service `drift-injector` tournait en mode démon 24/7 (toutes les 30 min), injectant des articles majoritairement fake (jusqu'à 90 %) en continu depuis des semaines. Résultat : 16 229 des 19 576 documents de la collection MongoDB (83 %) et autant sur Elasticsearch étaient des artefacts de simulation (sources synthétiques 'InfoBrûlante', 'VéritéCachée', 'AlerteComplot', etc., identifiables par le préfixe d'URL `https://simulation.test/`), faisant artificiellement grimper le taux de fake global affiché à ~80 %. Ces documents ont été supprimés des deux bases ; le taux de fake résiduel réel est revenu à ~52 %. Le service `drift-injector` est désormais désactivé par défaut (`docker-compose.yml`) : le drift ne se déclenche plus qu'à la demande, depuis le dashboard ou l'API.

● Seuil de décision relevé de 0.65 à 0.75

En complément de la correction du bug de format, le seuil de classification `p_fake` a été relevé de 0.65 à 0.75 dans `nlp_classifier.py` et `web_search.py`, réduisant davantage les faux positifs sur les dépêches d'agences officielles.

● Le drift n'est plus un état permanent — fenêtre de visualisation puis retour automatique

Auparavant, la phase de « récupération » suivait l'injection de drift avec seulement 5 secondes de délai, rendant le drift quasiment invisible dans Grafana/Streamlit avant sa propre correction. `scripts/inject_drift_simulation.py` introduit maintenant un paramètre `visualization_window` (60-600 s, 300 s = 5 min par défaut) : le drift reste actif et observable pendant cette fenêtre, puis des articles réels sont envoyés automatiquement pour ramener le modèle et les statistiques à la normale — sans intervention manuelle. Le endpoint POST `/api/v1/drift/inject` expose ce paramètre ; l'interface Streamlit propose un curseur (1-10 min) pour le régler. Testé de bout en bout : cycle complet drift → fenêtre → récupération automatique validé.

● **Enrichissement du corpus d'entraînement — contexte africain multilingue**

Le script `scripts/download_african_datasets.py` a été exécuté pour enrichir `data/raw/africa_news/africa_news.csv` : ajout de 2 742 articles réels MasakhaNEWS couvrant 11 langues africaines (amharique, haoussa, igbo, lingala, oromo, pidgin nigérian, shona, somali, swahili, tigrinya, yoruba) ainsi que des articles récents via Google News RSS (sources africaines). Le corpus africain total passe de 854 à 3 601 exemples. Limite constatée : Africa Check (source de fact-checks africains) est inaccessible depuis l'environnement de développement (blocage Cloudflare 403) ; le volume de fake news africaines authentiques reste donc limité (~77 exemples), documenté comme limite connue pour les travaux futurs. `scripts/train_model.py` applique un facteur de sur-pondération (`--africa_boost`, ×5 par défaut) sur les exemples africains via `WeightedRandomSampler`, sans supprimer aucune donnée existante (`welfake`, `isot`, `liar`, `fakenewsnet` conservés intégralement).

● **Ré-entraînement complet du modèle Continual-DistilBERT**

Un nouveau cycle d'entraînement complet a été lancé sur le corpus enrichi (55 356 exemples d'entraînement, 50/50 fake/réel). Le script `train_model.py` a été rendu résilient aux coupures (sauvegarde d'un checkpoint de reprise tous les 100 batchs, argument `--save_every`) suite à des interruptions dues à des déconnexions intempestives du disque externe pendant l'entraînement (voir point suivant).

● **Stabilisation Docker — instabilité récurrente du disque externe, corrigée de façon permanente**

Le disque externe hébergeant le projet s'est déconnecté/reconnecté à de multiples reprises pendant cette session (points de montage successifs `local5` → `local6` → `local7` → `local8` → `disqueD`), cassant à chaque fois les points de montage (`bind mounts`) Docker. Diagnostic confirmé via `journalctl/dmesg` : véritables erreurs matérielles SATA ("`ata1.00: error: { UNC }`"), pas un simple problème de configuration logicielle. Cause racine du changement de nom de dossier : la partition n'était déclarée nulle part dans `/etc/fstab`, donc `udisks2/GNOME` la remontait automatiquement sous un nouveau nom à chaque reconnexion. Correction définitive apportée : ajout d'une entrée fixe dans `/etc/fstab` basée sur l'UUID de la partition (et non plus sur un nom généré dynamiquement), avec l'option `'nofail'` pour ne jamais bloquer le démarrage. Le pilote noyau `ntfs3` s'étant révélé incompatible avec le volume après une déconnexion brutale (erreur `'bad superblock'`), le montage a été basculé sur le pilote `ntfs-3g` (FUSE), qui s'est montré fiable. Le point de montage est désormais fixe et permanent : `/mnt/disqueD/ISTIN/MASTER/M2/S1/PROJET TUTEUR/désinformation-pipeline`. Les volumes nommés Docker (`mongodb_data`, `elasticsearch_data`) n'ont à aucun moment été affectés — aucune perte de données sur toute la session malgré les multiples coupures.